

Dynamika objektů

Karel Richta a kol.

Katedra technických studií
Vysoká škola polytechnická Jihlava

© Karel Richta, 2021

Objektově-orientované programování, OOP
03/2021, Lekce 3

<https://moodle.vspj.cz/course/view.php?id=201479>



Datové typy C++

- Základní datové typy:
 - celočíselné: short, int, long
 - reálné: float, double, long double
 - znaky a řetězce: char (znak) a pole znaků char p[10]; (hlavně v C)
 - předdefinované řetězce – string (v C++)
 - prázdný typ: void
- Speciální, strukturované:
 - pole – int pole[10] – homogenní struktura
 - výčtový typ, enum barva{R,G,B};
 - struktura (struct) – heterogenní struktura (hlavně v C)
 - třída (class) – heterogenní struktura (v C++)

Opakování: Globální a lokální proměnné

- Proměnné jsou lokální a globální (deklarace, definice(inicializace)).
- Lokální proměnná má platnost v bloku.
- Globální proměnná má platnost ve všech funkcích a metodách (může ji zakrýt lokální deklarace).
- V C++ lze pro určení globální proměnné použít operátor čtyřtečky :: .

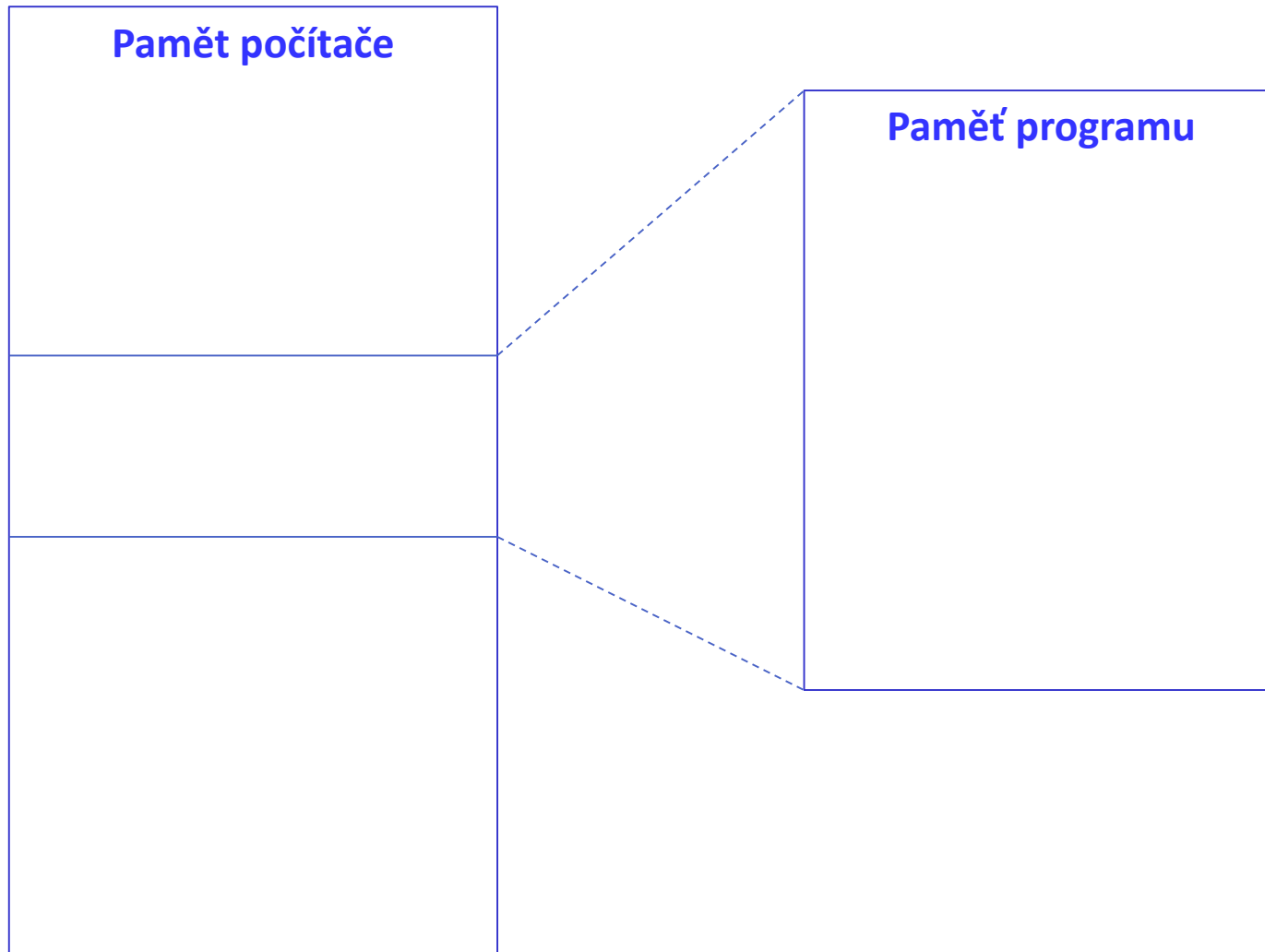
```
int a = 0, b = 0, c = 1; /* globální proměnné */
void funkce(int a, int b) //formální argumenty {
//int a = -5; lokální proměnné - zde nelze předefinovat
    cout << "main: a = " << a << ", b = " << b << ", c = " << c
<< endl;
}
int main(void) {
    int c = 25;
    cout << "main: a = " << a << ", b = " << b << ", c = " << c <<
endl;
    funkce(100, 200);
    return 0;
}
```

Dynamické proměnné

- Kromě lokálních a globálních objektů (statických proměnných) existuje ještě jedna možnost, jak objekty dynamicky vytvářet – na tzv. haldě (heap).
- Objekty na haldě vytváří v C++ explicitně programátor – pomocí operátoru **new**.
- Ruší je naopak pomocí operátoru **delete**.
- Operátory **new** a **delete** využívají konstruktory a destruktory – ty probereme za chvíli.
- Příklad:

```
int main () {  
    int *px = new int;  
    *px = 7;  
    delete px;  
    int *pp = new int[10];  
    for (int i = 0; i < 10; i++) *pp[i] = i;  
    delete [] pp;  
    return 0;  
}
```

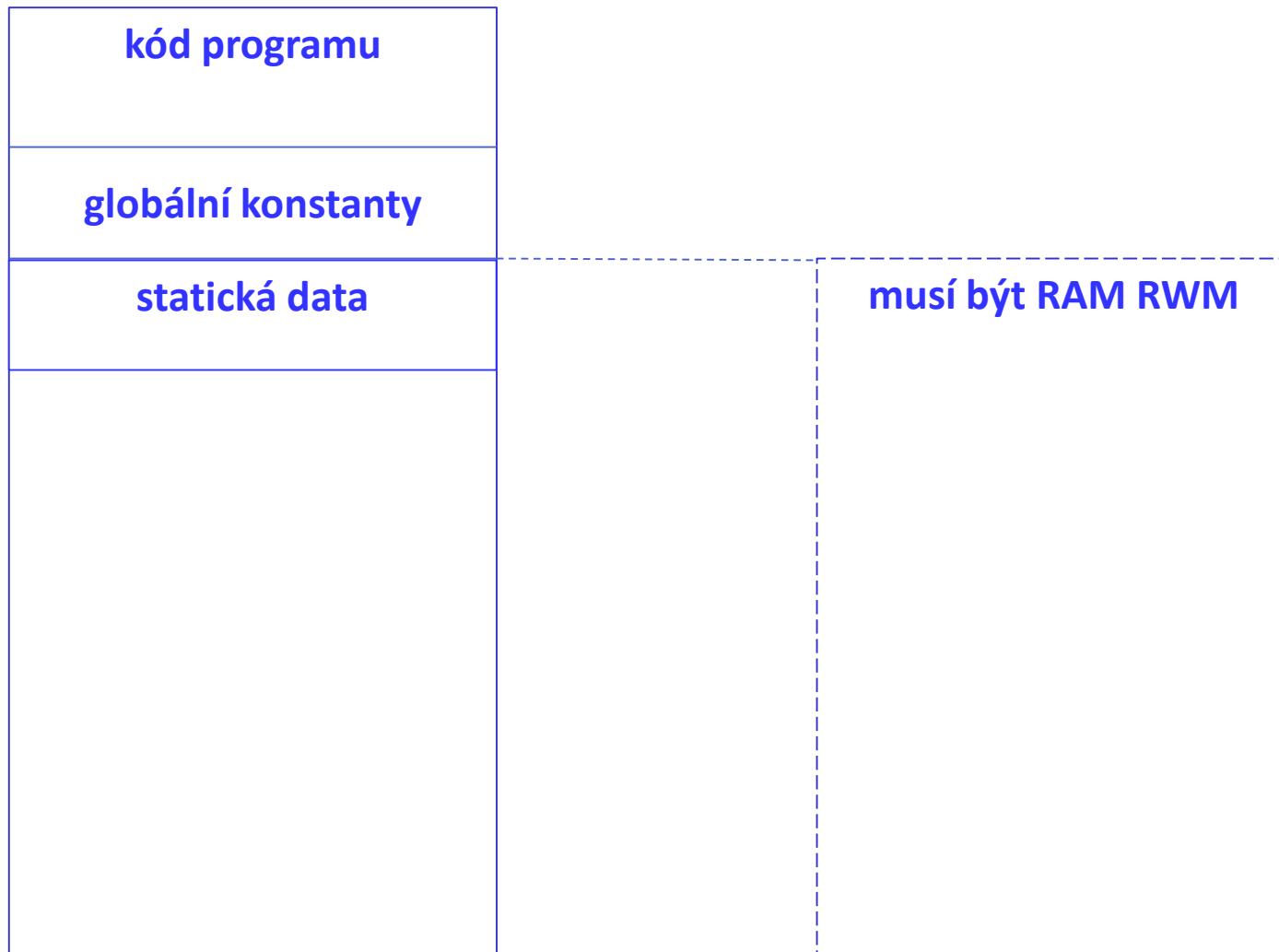
Správa paměti



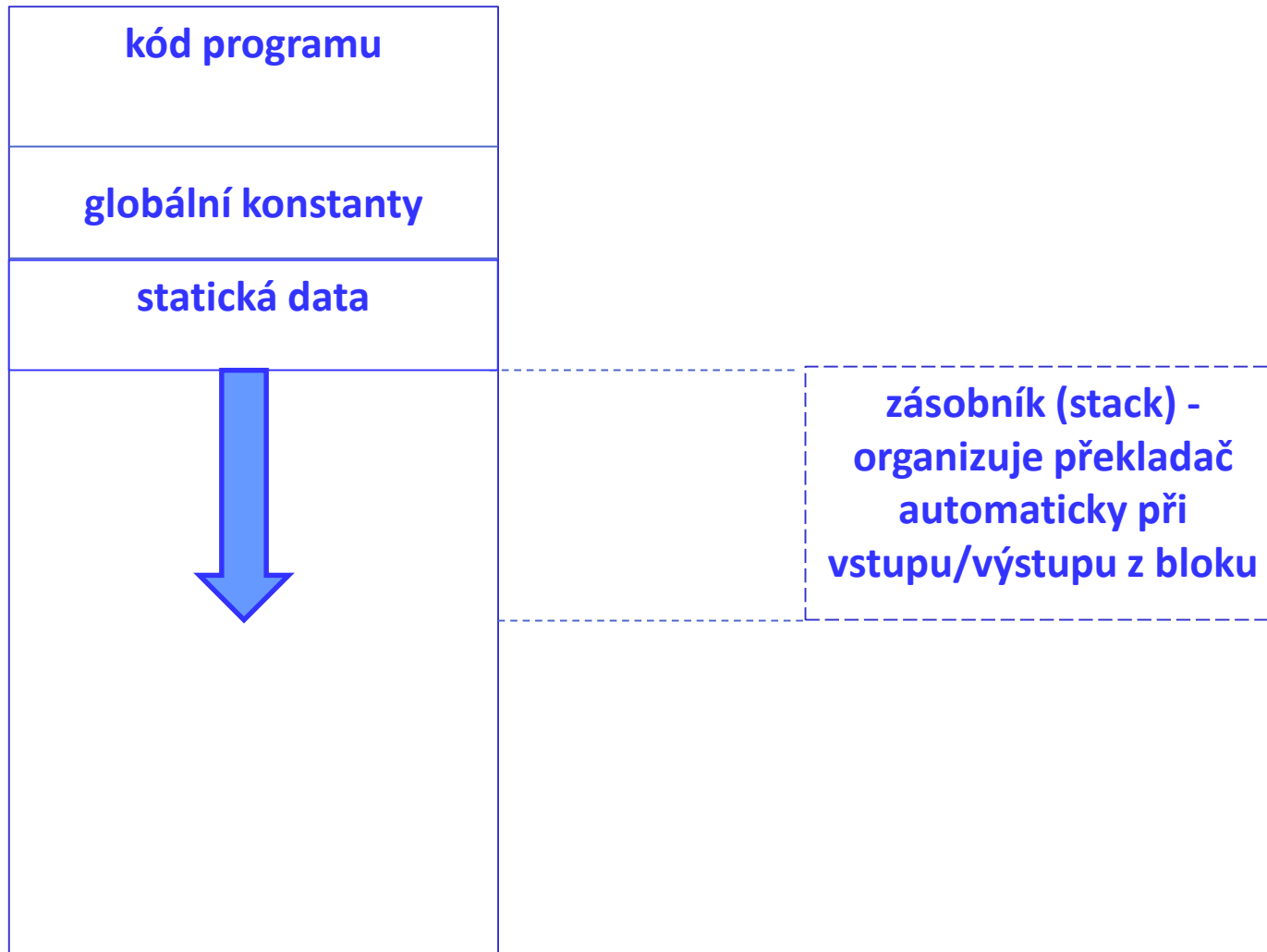
Paměť programu

kód programu		může být ROM (to může hlídat HW, nebo SW, nebo nikdo)
globální konstanty		

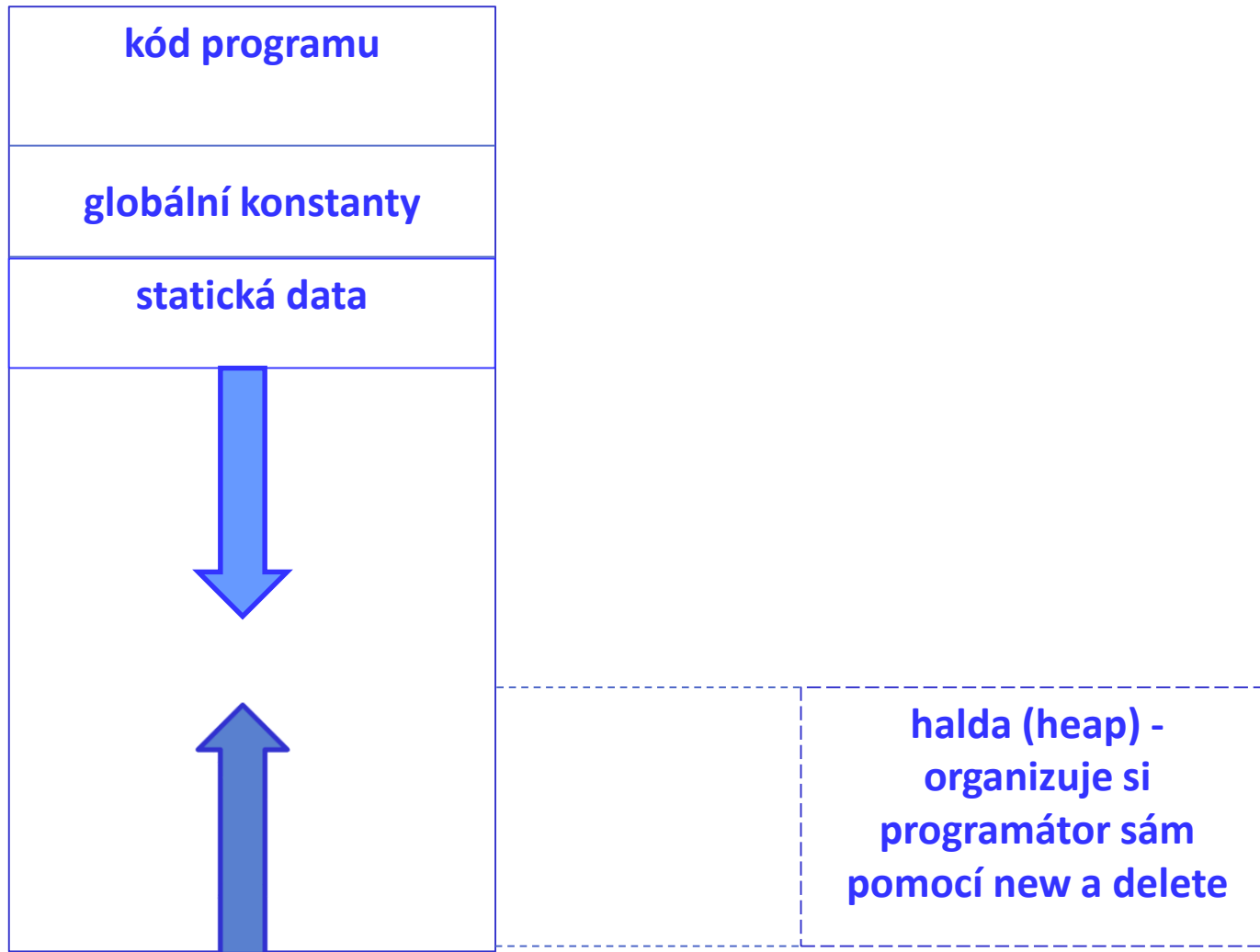
Statická data



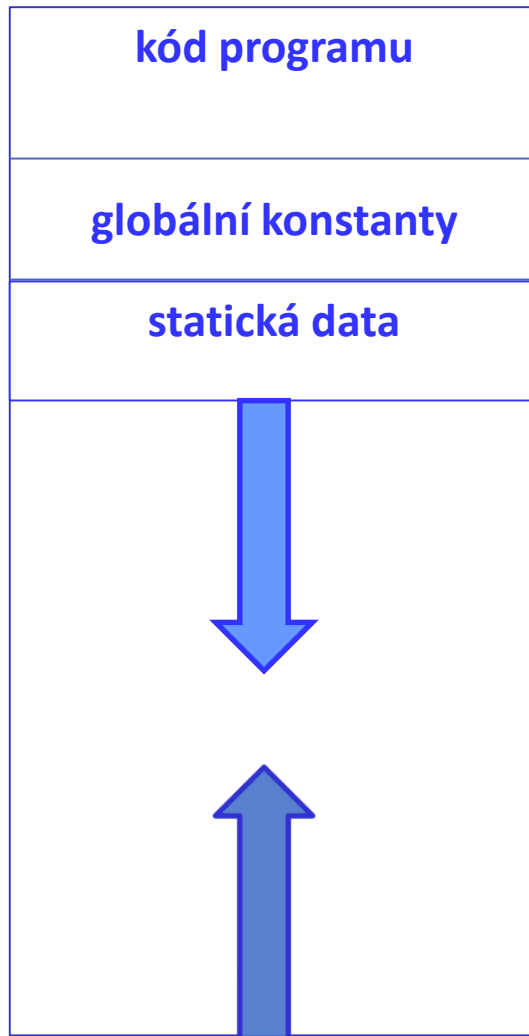
Dynamická data - zásobník



Dynamická data - halda

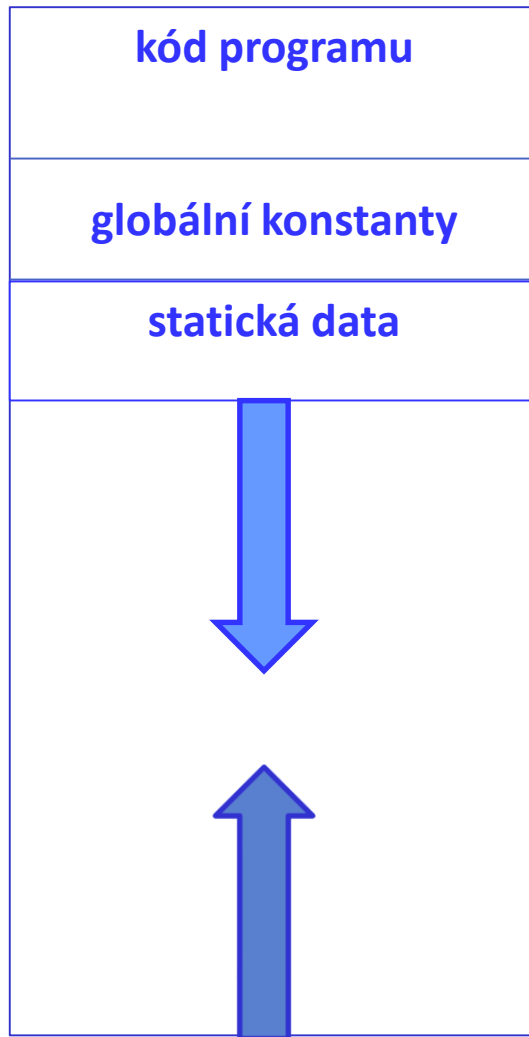


Jak umístíme objekt mezi gl.konstanty



```
const int x = 10;
```

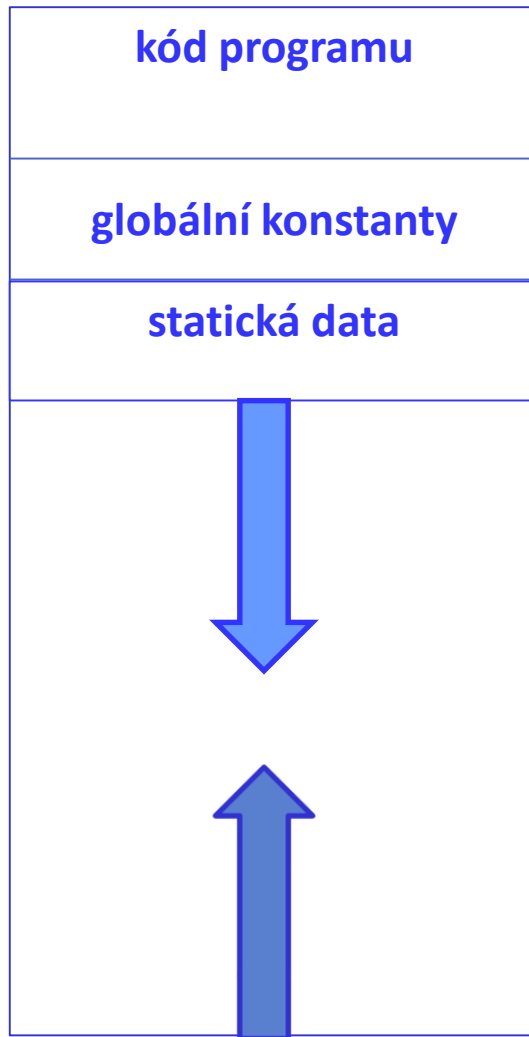
Jak umístíme objekt mezi statická data



```
const int x = 10;
```

```
int y;  
int z = 10;
```

Jak umístíme objekt na zásobník

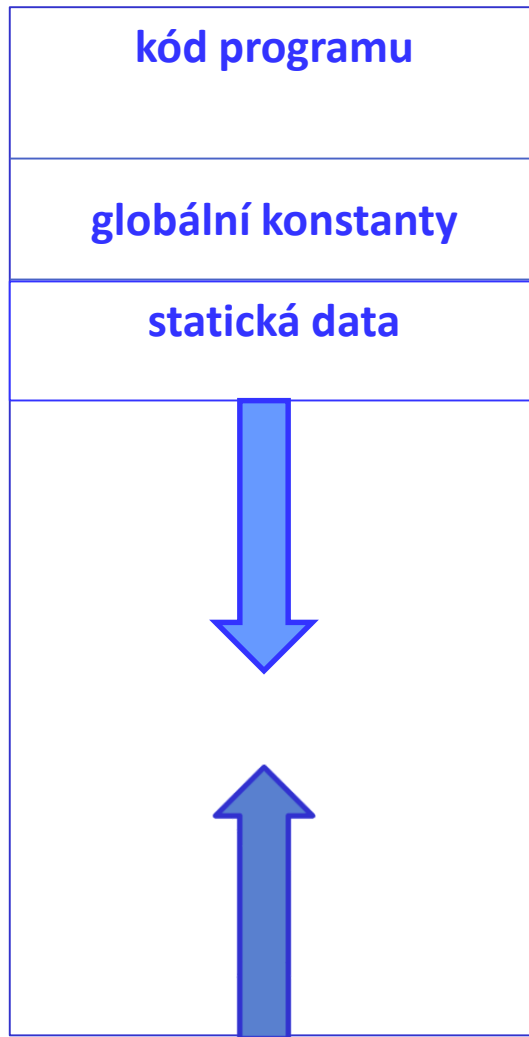


```
const int x = 10;
```

```
int y;  
int z = 10;
```

```
{ int w;  
  int u = 10;  
}
```

Jak umístíme objekt na haldu



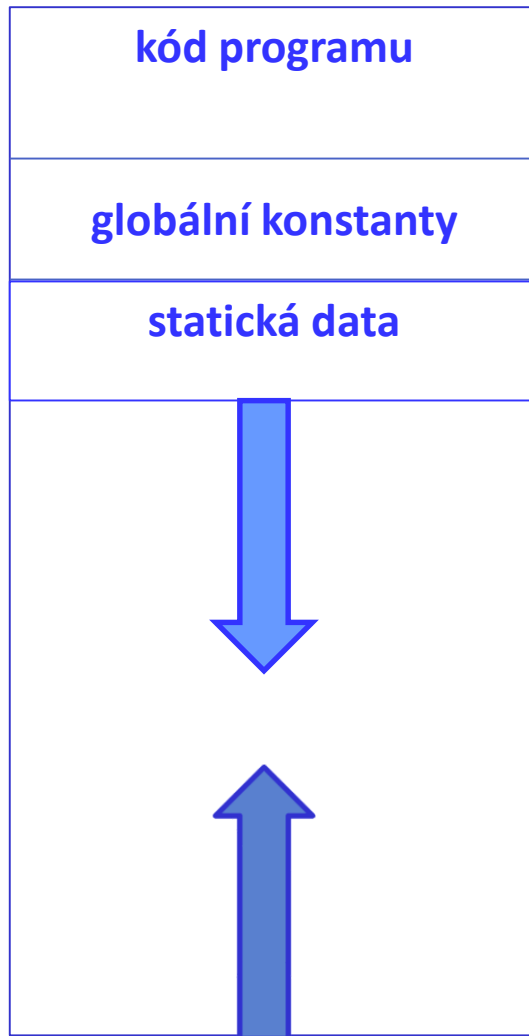
```
const int x = 10;
```

```
int y;  
int z = 10;
```

```
{ int w;  
  int u = 10;  
}
```

```
int *s = new int;  
int *t = new int(10);
```

Jak zrušíme objekty



```
const int x = 10;
```

když skončí
program

```
int y;  
int z = 10;
```

když skončí
program

```
{ int w;  
  int u = 10;  
}
```

když skončí blok

```
int *s = new int;  
int *t = new int(10);
```

```
delete s;  
delete t;
```

tady to musí zařídit
programátor

Opakování: Třídy a jejich instance

- Objekty s podobnými vlastnostmi sdružujeme do tříd.
- Příslušnost objektu do třídy vyjadřujeme tvrzením, že objekt je instancí dané třídy (termíny objekt a instance třídy jsou synonyma).
- Třída je „forma“ na vytváření svých instancí.
- Třída = zobecněný klasický datový typ:
 - Vedle množiny hodnot definuje i množinu přípustných operací nad těmito hodnotami - třída definuje vlastnosti a schopnosti svých instancí.

Základní pilíře OOP

- **Zapouzdření** (kód je pohromadě se zpracovávanými daty):
 - Skrývání implementace (nikdo nemá mít šanci zjistit, jak je program implementován).
 - Zvýšení bezpečnosti a robustnosti (nemožnost nekorektního použití).
 - Usnadnění budoucích modifikací.
- **Polymorfismus:**
 - Každá zpráva musí mít svého adresáta, nelze ji poslat „do prostoru“.
 - Objekt sám rozhodne, jak na zprávu zareaguje.
- **Skládání:**
 - Objekt může obsahovat jiné objekty.
- **Dědičnost:**
 - Speciální případ, při němž s objektem převezmou i jeho rozhraní.
 - Omezuje duplicity v kódu.
 - Nebezpečí špatného použití (narušuje zapouzdření).
 - Používání návrhových vzorů.
 - Proč bych měl vymýšlet něco, co už je vymyšlené, a je to vymyšlené dobře.

Deklarace třídy a definice metod

Deklarace

```
class NazevTridy {  
    deklarace clenskych dat;  
    prototypy clenskych metod;  
    prototypy konstruktoru;  
    prototyp destruktoru;  
};
```

```
class Trida {  
    int a;  
    void tisk();  
    Trida();  
    ~Trida();  
};
```

Definice (metod)

```
navratova_hod  
    NazevTridy::metoda(argumenty){  
        . . .  
    }
```

```
void Trida::tisk() {  
    cout<<this->a;  
}
```

Členská data třídy

Datové položky třídy:

- Popisují datový typ položek, které u třídy definujeme.
- Standardní je jejich nepřístupnost mimo metody třídy => zapouzdření dat.

K určení přístupu ke členským datům slouží :

- **Private** (soukromé) – pouze veřejné členské metody daného objektu mohou přistoupit k položkám, přes identifikátor objektu a tečkovou notaci.
- **Public** (veřejné) – libovolný program může přistupovat k položkám, přes identifikátor objektu a tečkovou notaci.
- Doporučení: členská data vždy v části private => ukrývání dat.

```
class Auto{
private:
    string spz;
    int objem;
public:
    void setSpz(const char
*spz) ;
    void setObjem(int obj );
};
```

```
int main() {
    Auto a1;
    a1.spz = "1000"; // nelze
    a1.setSpz("ABC-20-10");
    a1.setObjem(100); // nelze
                        // a1.objem =
100
    return 0;
}
```

Členské metody třídy

- V deklaraci třídy se uvádějí prototypy členských metod.
- V deklaraci i definici musí být uvedený návratový typ a případné argumenty.
- Členské metody mohou přistupovat k soukromým členským datům i metodám.
- Členské metody v části **public**, jsou přístupné ze všech částí programu přes identifikátor objektu a tečkovou notaci a tvoří rozhraní třídy (deklarace by se neměla měnit, implementace ano).
- Metody, které nechceme zpřístupnit jsou **private**.
- Členské metody v části **private** jsou přístupné pouze z ostatních členských metod (pomocné služby).
- Definice není zakončena středníkem, deklarace je.
- Zavolání členské metody = zaslání zprávy objektu = zpřístupnění členských dat třídy tomu objektu, který zprávu vyvolal.

```
int Auto::getObjem() { return objem; }  
string Auto::getSpz() { return spz; }
```

Vložené členské metody (definice v deklaraci)

- Metoda definovaná v deklaraci třídy je vložená.
- Přeložená **inline** metoda je při překladu vložena na místo volání, nedochází ke skoku na jinou adresu (doporučený postup).
- Mimo deklaraci je nutné v definici použít klíčové slovo **inline**.
- **Inline** překlad není zaručen.
- Definice **inline** funkce uvnitř deklarace není zakončena středníkem.

```
class Auto{
string spz;
    int objem;
public:
    void setSpz(const char *spz) {
        return spz; }
    void setObjem(int obj ) {
        objem_ = obj; }
    int getObjem();
    string getSpz(){ return spz; }
};
inline int Auto::getObjem() {
return objem; }
```

```
int main() {
    Auto a1, a2;
    a1.setSpz („6A6-10-10“);
    a1.setObjem(1500);
    a2.setSpz („8DD-20-20“);
    a2.setObjem(1000);
    cout << a2.getObjem() << " "
        <<a1.getObjem();
    return 0;
}
```

Konstruktory

- Konstruktor můžeme vnímat jako metodu, která se jmenuje stejně jako třída, jejíž instanci vytváří a typ návratové hodnoty se neuvádí – je to nová instance třídy.
- Vytvoření instance třídy (objektu) je shodné s vytvořením proměnné uživatelského typu ----- **Auto a1**;
- Motivace tříd – činnost jako s datovým typem, ale:
 - **a1 = {10, "10-BAC-20"}; // nelze**
- Nutnost inicializační členské metody, která bude zavolána dříve než ostatní metody, tj. první = konstruktor.
- Konstruktor je automaticky volán při vytvoření objektu.
- Konstruktor – členská metoda = prototyp i definici.
- Konstruktoru nesmí mít návratový typ – ani **void**.
- Konstruktor má stejné jméno jako třída.
- Konstruktor nelze vyvolat přes tečkovou notaci.

Příklad

```
class Auto{
private:
    int objem; string spz;
public:
    void initializeAuto(string s, int obj) {
        spz = s; objem = obj;
    }
    Auto(string s, int obj) {
        spz = s; objem_ = obj;
    }
};

int main() {
    Auto a1 = Auto("10-ABC-10",1500);
        //a1.initializeAuto("10-ABC-10",1500);
    Auto a2("00-YYY-00",1000);
    cout << a1.getSpz() << " " << a2.getSpz();
    return 0;
}
```

[Příklady](#)

Konstruktor – členská metoda

- Nemusí být vložený, lepší je oddělit jeho deklaraci a definici.
- Zajišťuje konzistenci objektu.
- Může být přetížen – může existovat více konstruktorů pro stejnou třídu. Nemohou se lišit jménem (to je stejné), musí se ale lišit argumenty.
- Standardně použit pro inicializaci členských dat.

```
class Auto{
private:
    int objem; string spz;
public:
    string getSpz () { return spz; }
    Auto(string s, int obj);
};
Auto::Auto(const char * spz, int obj){ strcpy(spz_,spz);objem_ = obj; }
int main() {
    Auto a1 = Auto("10-ABC-10",1500); Auto a2("00-YYY-00",1000);
    cout << a1.getSpz() << " " << a2.getSpz();
    return 0;
}
```

Implicitní konstruktor (default)

- Implicitní konstruktor je vytvořen automaticky překladačem.
- Je použit pro inicializaci objektu u kterého nebyla zadána žádná inicializační data.
- Je to konstruktor bez argumentů.
- Odpovídá vytvoření proměnné bez její inicializace: `int i;`
- Obdobně: `Auto a1;`

```
class Auto {  
private:  
    int objem; string spz;  
public:  
    Auto(string s, int obj) { spz = s; objem = obj; }  
    string getSpz() { return spz; }  
    void setSpz(string s) { spz = s; }  
    Auto() {};// Implicitni konstruktor  
};
```

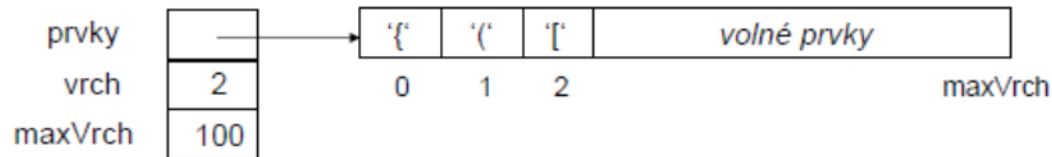
Přetížení konstruktorů

- Konstruktory lze přetěžovat, musí se ale poznat, který se má vyvolat.
- Pokud vytvořím vlastní variantu konstruktoru (explicitní) – překladač nevytvoří implicitní – je vhodné ho dotvořit vlastními prostředky – doporučení vždy vytvořit.
- Implicitní konstruktor lze řešit jako variantu explicitního konstruktoru s implicitními argumenty.

```
class Auto {  
private:  
    int objem; string spz;  
public:  
    Auto(string s, int obj) { spz = s; objem = obj; }  
    string getSpz() { return spz; }  
    void setSpz(string s) { spz = s; }  
    Auto() { spz = "00-000-00"; objem = 0; }; // Implicitni konstruktor  
    //Auto(string s = "00-000-00", int obj = 0) { spz = s; objem = obj; }  
};
```

Konstruktor – inicializační sekce

- Přímá inicializace členských dat.
- Nutné pro inicializaci členských dat s referenčním typem a pro konstanty.
- Rychlost – nevytváří se lokální kopie argumentů.
- Použito též při předání argumentů pro konstruktor rodiče.



```
Zasobnik::Zasobnik(int maxDelka) :  
    maxVrch(maxDelka-1),  
    vrch(-1) {  
    prvky = new TypPrvku[maxDelka];  
}
```

Pozor: Pořadí inicializace je dáno deklarací, nikoliv pořadím v seznamu inicializátorů.

```
class Zasobnik {  
public:  
    Zasobnik(int maxDelka = 100);  
    ~Zasobnik();  
    void vloz(TypPrvku p);  
    TypPrvku odeber();  
    bool jePrazdny();  
private:  
    TypPrvku *prvky;  
    int vrch;  
    int maxVrch;  
};
```

Možné způsoby vzniku objektu (1/2)

<pre>Obj o; Obj o{}; Obj* o = new Obj; Obj* o = new Obj(); Obj* o = new Obj{};</pre>	V těchto případech se zavolá výchozí konstruktor.
<pre>Obj o = 42; Obj o(42); Obj o{42}; Obj* o = new Obj(42); Obj* o = new Obj{42};</pre>	V těchto případech se zavolá konstruktor, který přijímá parametr typu <code>int</code>.
<pre>Obj o[42]; Obj o[42]{}; Obj* o = new Obj[42]; Obj* o = new Obj[42](); Obj* o = new Obj[42]{};</pre>	V těchto případech se 42-krát zavolá výchozí konstruktor.
<pre>Obj o();</pre>	Zrada (známá jako <i>most vexing parse</i>): toto není vznik objektu, ale deklarace funkce.

Možné způsoby vzniku objektu (2/2)

<pre>void receivesObj(Obj o); ... receivesObj({}); // výchozí receivesObj(42); // přijímající int</pre>	Vznik objektu až při předávání hodnoty do funkce
<pre>Obj returnsObj() { ... return {}; // výchozí return 42; // přijímající int</pre>	Vznik objektu při vracení hodnoty z funkce
<pre>... Obj() ...; // výchozí ... Obj{} ...; // výchozí ... Obj(42) ...; // přijímající int ... Obj{42} ...; // přijímající int</pre>	Vznik dočasného (<i>temporary</i>) objektu – objekt nemá název
<pre>struct Foo { Obj o; Foo() : o(), // výchozí o{}, // výchozí o(42), // přijímající int o{42}, // přijímající int chybí // podle deklarace</pre>	Vznik objektu v inicializačním seznamu konstruktoru

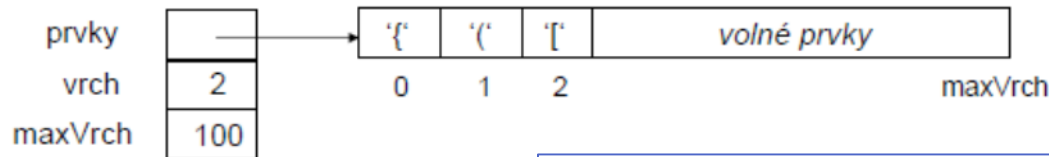
Destruktor (*destructor*)

- Vždy, když zanikne instance nějaké třídy, je skrytě (implicitně) zavolána metoda zvaná **destruktor**.
- Destruktor třídy T je její metoda pojmenovaná ~T.
- Rozdíly oproti konstruktoru:
 - Nemá parametry.
 - Nemá inicializační seznam.
 - V dané třídě může být pouze jeden.

```
struct Person {  
    ~Person() {  
        std::cout << "destruktor\n";  
    }  
};
```

Destruktor

- Destruktor nelze volat explicitně v programu.
- Je volán při ukončení platnosti objektu:
 - Objekt v těle main – na konci programu.
 - Dynamicky objekt – řídí uživatel (operátory new a delete).
 - Automatický objekt – ukončen na konci bloku, kde byl definován.



```
Zasobnik::~~Zasobnik() {  
    delete [] prvky;  
    maxVrch = -1;  
    prvky = NULL;  
    vrch = -1;  
}
```

```
class Zasobnik {  
public:  
    Zasobnik(int maxDelka = 100);  
    ~Zasobnik();  
    void vloz(TypPrvku p);  
    TypPrvku odeber();  
    bool jePrazdny();  
private:  
    TypPrvku *prvky;  
    int vrch;  
    int maxVrch;  
};
```

Význam destruktoru

- Provádí úklid; uvolňuje **prostředky**, za které má daná instance **odpovědnost**.
 - Prostředky mohou být: paměť, soubory, připojení k databázím, a další.

```
struct User {  
    User() : data(new UserData) { // získkej paměť  
        systemLogIn(*this);      // přihlas se  
    }  
    ~User() {  
        systemLogOut(*this);     // odhlas se  
        delete data;             // uvolni paměť  
    }  
private:  
    UserData* data;  
};
```

Příklad 1

```
>>> int main() {  
    int foo;  
}
```

main() = ?

Příklad 1

```
int main() {  
>>>     int foo;  
}
```

main() = ?

Příklad 1

```
int main() {  
>>>     int foo;  
}
```

foo = ?

main() = ?

Příklad 1

```
int main() {  
    int foo;  
>>> }
```

foo = ?

main() = ?

Příklad 1

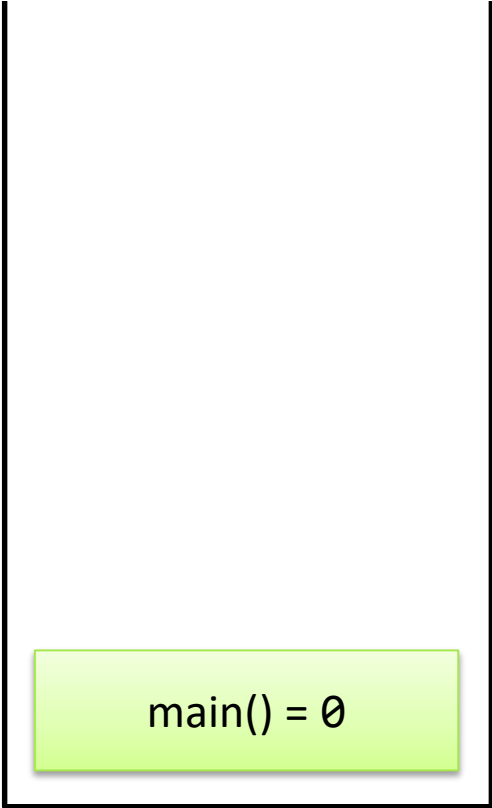
```
int main() {  
    int foo;  
>>> }
```

foo = ?

main() = 0

Příklad 1

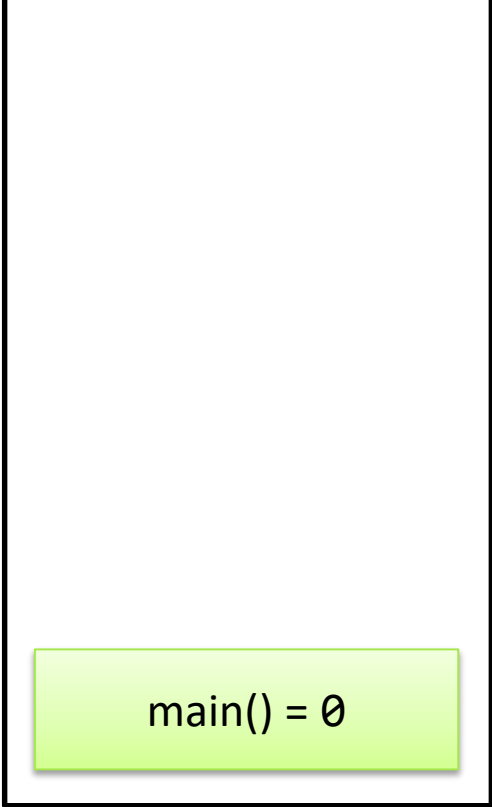
```
int main() {  
    int foo;  
>>> }
```



main() = 0

Příklad 1

```
int main() {  
    int foo;  
}
```



main() = 0

Příklad 2

```
>>> int main() {  
    int i = 0;  
    while (i < 2) {  
        std::string str;  
        i++;  
    }  
}
```

main() = ?

Příklad 2

```
int main() {  
>>>   int i = 0;  
      while (i < 2) {  
          std::string str;  
          i++;  
      }  
}
```

main() = ?

Příklad 2

```
int main() {  
>>>   int i = 0;  
      while (i < 2) {  
          std::string str;  
          i++;  
      }  
}
```

i = 0

main() = ?

Příklad 2

```
int main() {  
    int i = 0;  
>>> while (i < 2) {  
        std::string str;  
        i++;  
    }  
}
```

i = 0

main() = ?

Příklad 2

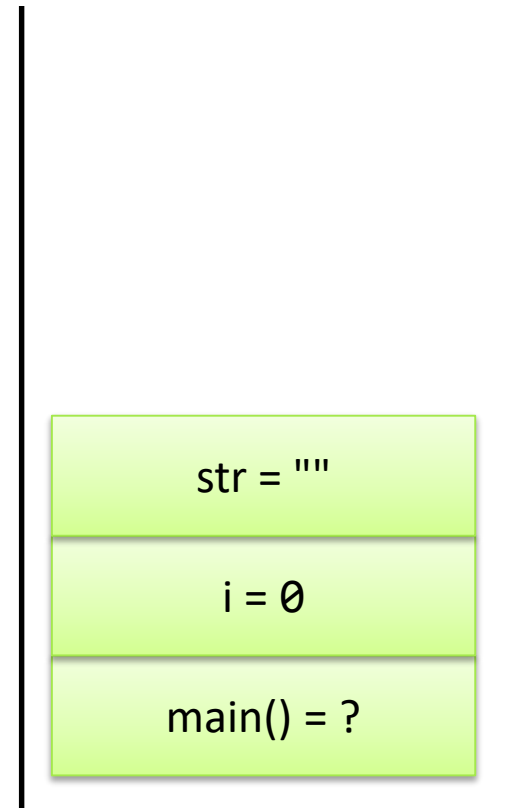
```
int main() {  
    int i = 0;  
    while (i < 2) {  
        >>>    std::string str;  
        i++;  
    }  
}
```

i = 0

main() = ?

Příklad 2

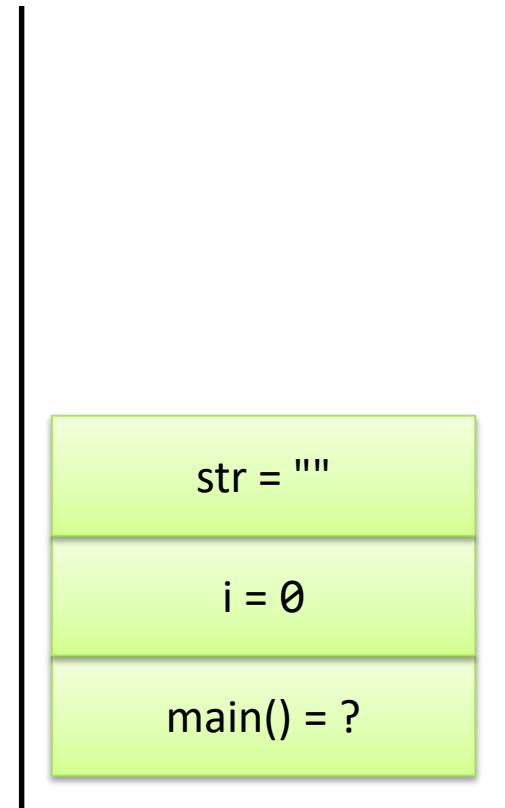
```
int main() {  
    int i = 0;  
    while (i < 2) {  
        >>>    std::string str;  
        i++;  
    }  
}
```



Příklad 2

```
int main() {  
    int i = 0;  
    while (i < 2) {  
        std::string str;  
        i++;  
    }  
}
```

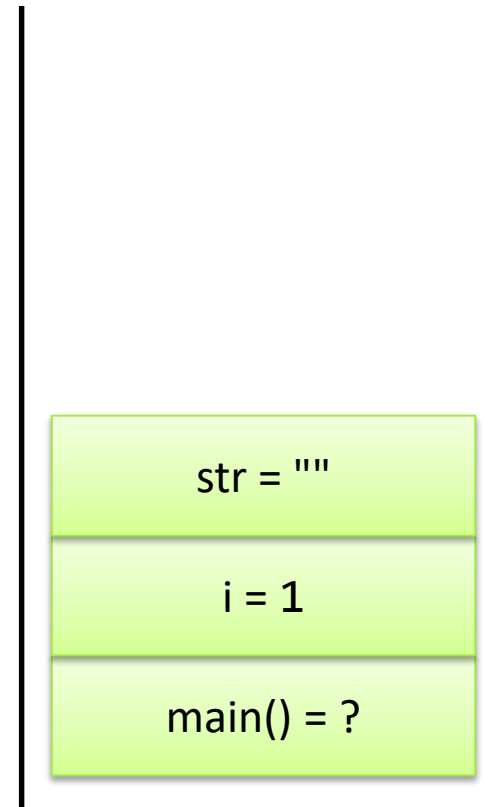
>>>



Příklad 2

```
int main() {  
    int i = 0;  
    while (i < 2) {  
        std::string str;  
        i++;  
    }  
}
```

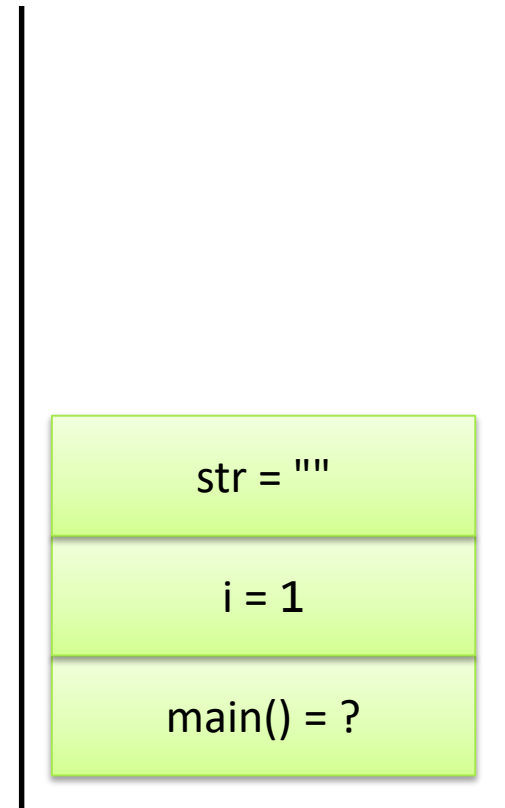
>>>



Příklad 2

```
int main() {  
    int i = 0;  
    while (i < 2) {  
        std::string str;  
        i++;  
    }  
}
```

>>>



Příklad 2

```
int main() {  
    int i = 0;  
    while (i < 2) {  
        std::string str;  
        i++;  
    }  
}
```

>>>

i = 1

main() = ?

Příklad 2

```
int main() {  
    int i = 0;  
>>> while (i < 2) {  
        std::string str;  
        i++;  
    }  
}
```

i = 1

main() = ?

Příklad 2

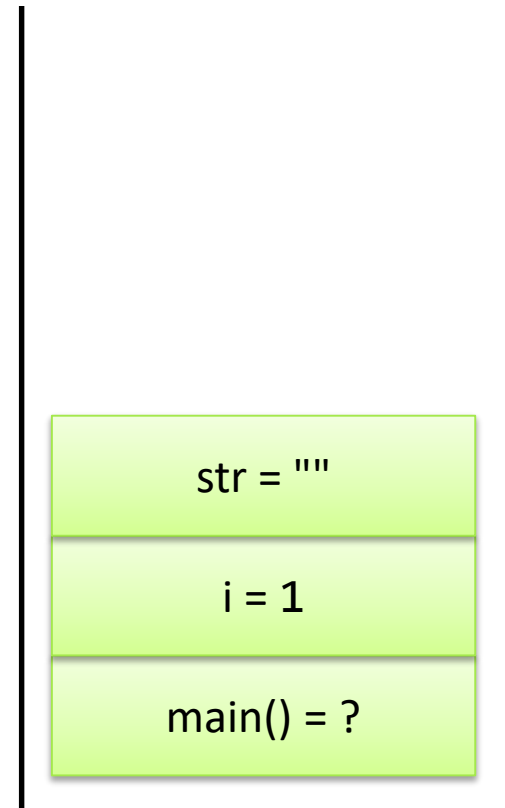
```
int main() {  
    int i = 0;  
    while (i < 2) {  
        >>>    std::string str;  
        i++;  
    }  
}
```

i = 1

main() = ?

Příklad 2

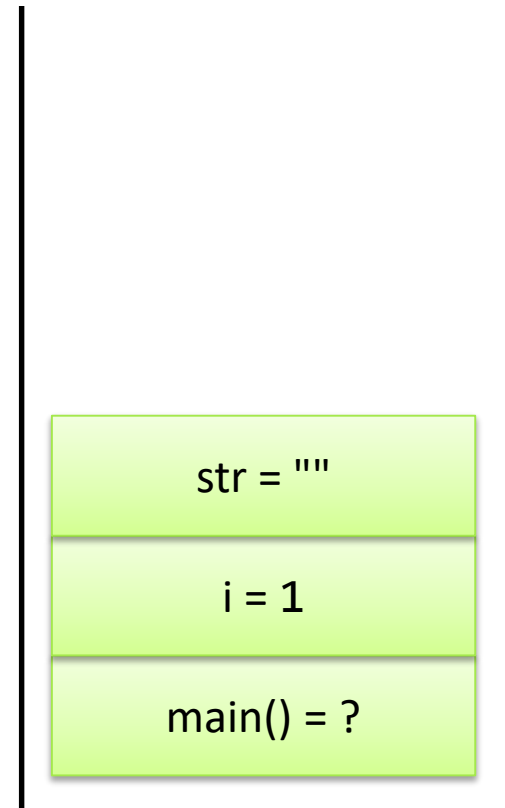
```
int main() {  
    int i = 0;  
    while (i < 2) {  
        >>>    std::string str;  
        i++;  
    }  
}
```



Příklad 2

```
int main() {  
    int i = 0;  
    while (i < 2) {  
        std::string str;  
        i++;  
    }  
}
```

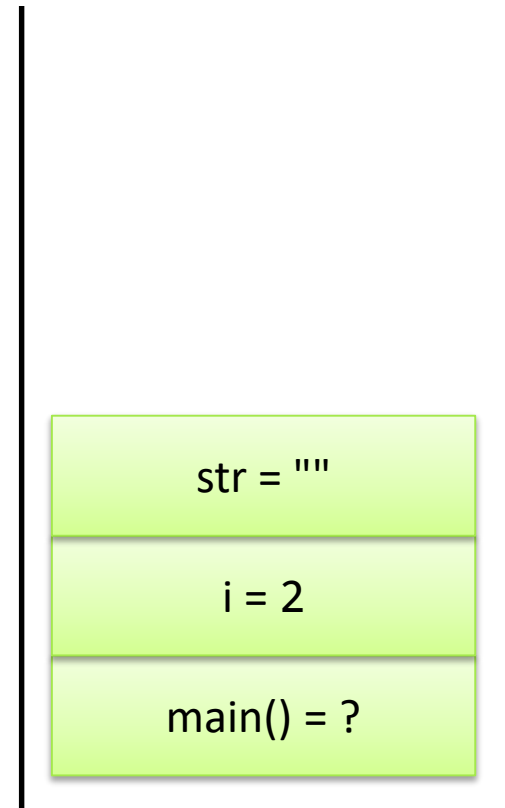
>>>



Příklad 2

```
int main() {  
    int i = 0;  
    while (i < 2) {  
        std::string str;  
        i++;  
    }  
}
```

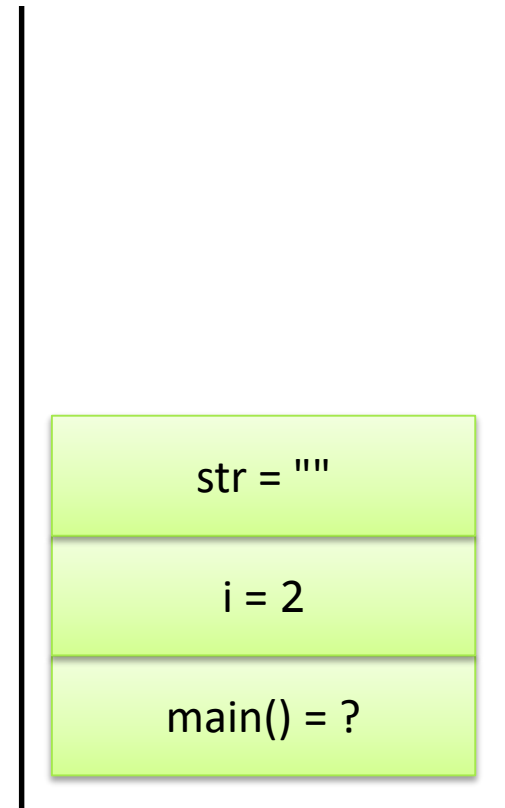
>>>



Příklad 2

```
int main() {  
    int i = 0;  
    while (i < 2) {  
        std::string str;  
        i++;  
    }  
}
```

>>>



Příklad 2

```
int main() {  
    int i = 0;  
    while (i < 2) {  
        std::string str;  
        i++;  
    }  
}
```

>>>

i = 2

main() = ?

Příklad 2

```
int main() {  
    int i = 0;  
>>> while (i < 2) {  
        std::string str;  
        i++;  
    }  
}
```

i = 2

main() = ?

Příklad 2

```
int main() {  
    int i = 0;  
    while (i < 2) {  
        std::string str;  
        i++;  
    }  
    >>> }
```

i = 2

main() = ?

Příklad 2

```
int main() {  
    int i = 0;  
    while (i < 2) {  
        std::string str;  
        i++;  
    }  
    >>> }
```

i = 2

main() = 0

Příklad 2

```
int main() {  
    int i = 0;  
    while (i < 2) {  
        std::string str;  
        i++;  
    }  
    >>> }
```

main() = 0

Příklad 2

```
int main() {  
    int i = 0;  
    while (i < 2) {  
        std::string str;  
        i++;  
    }  
}
```

main() = 0

Příklad 3

```
int func(int a){  
    int b = 2;  
    return b;  
}
```

```
>>>int main(){  
    int a = 10;  
    return func(a);  
}
```

main() = ?

Příklad 3

```
int func(int a){  
    int b = 2;  
    return b;  
}
```

```
>>> int main(){  
    int a = 10;  
    return func(a);  
}
```

main() = ?

Příklad 3

```
int func(int a){  
    int b = 2;  
    return b;  
}
```

```
>>> int main(){  
    int a = 10;  
    return func(a);  
}
```

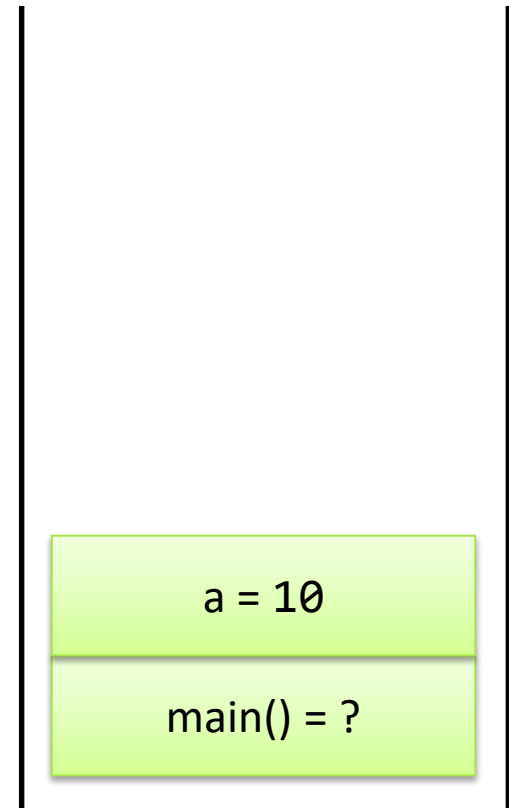
a = 10

main() = ?

Příklad 3

```
int func(int a){  
    int b = 2;  
    return b;  
}
```

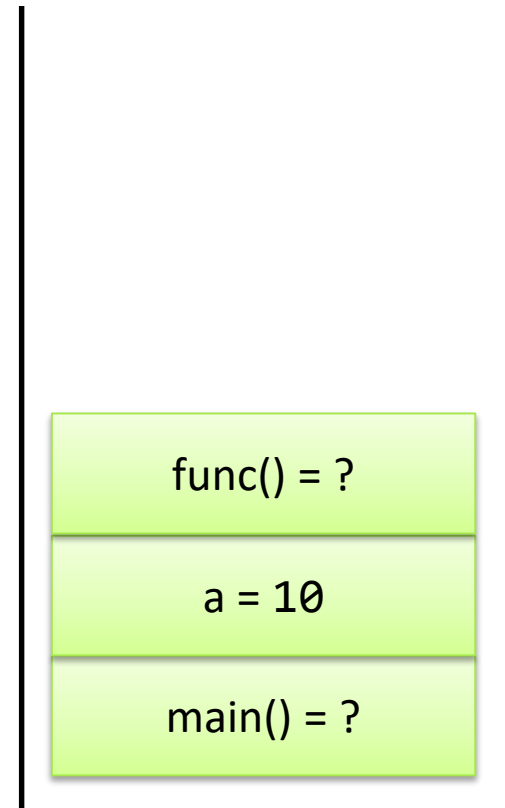
```
>>> int main(){  
    int a = 10;  
    return func(a);  
}
```



Příklad 3

```
int func(int a){  
    int b = 2;  
    return b;  
}
```

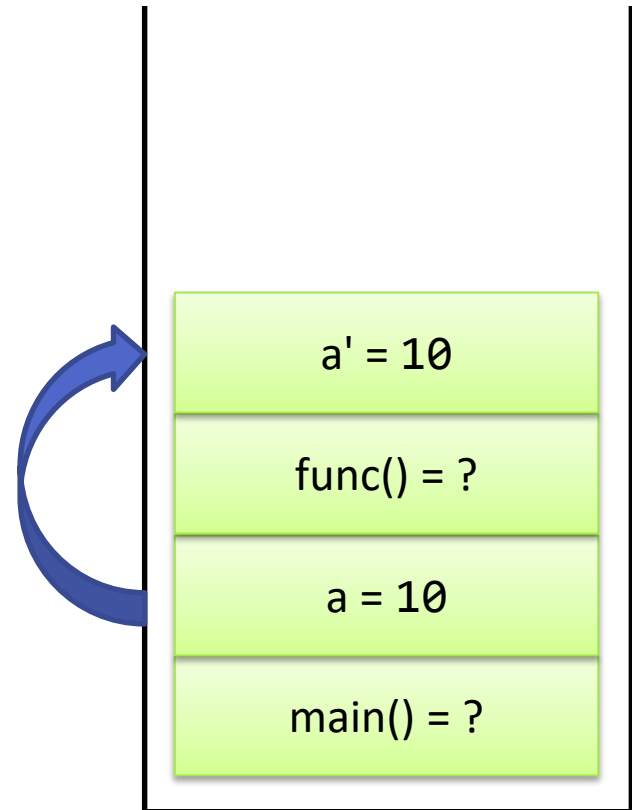
```
>>> int main(){  
    int a = 10;  
    return func(a);  
}
```



Příklad 3

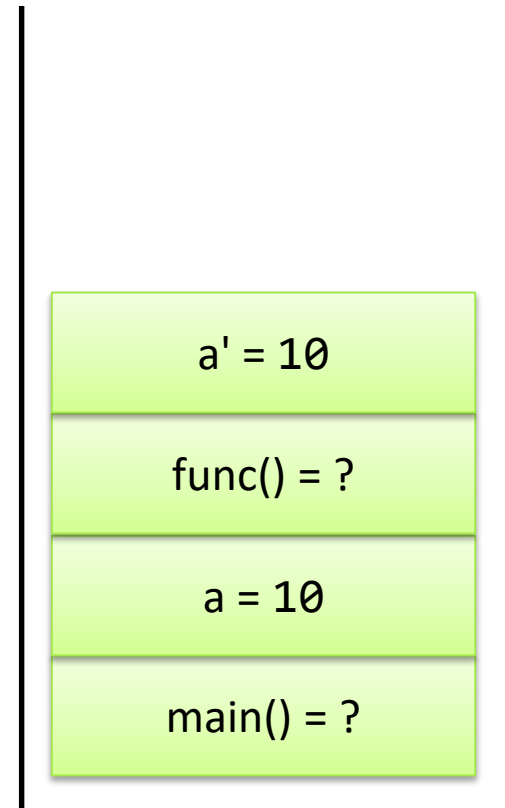
```
int func(int a){  
    int b = 2;  
    return b;  
}
```

```
>>> int main(){  
    int a = 10;  
    return func(a);  
}
```



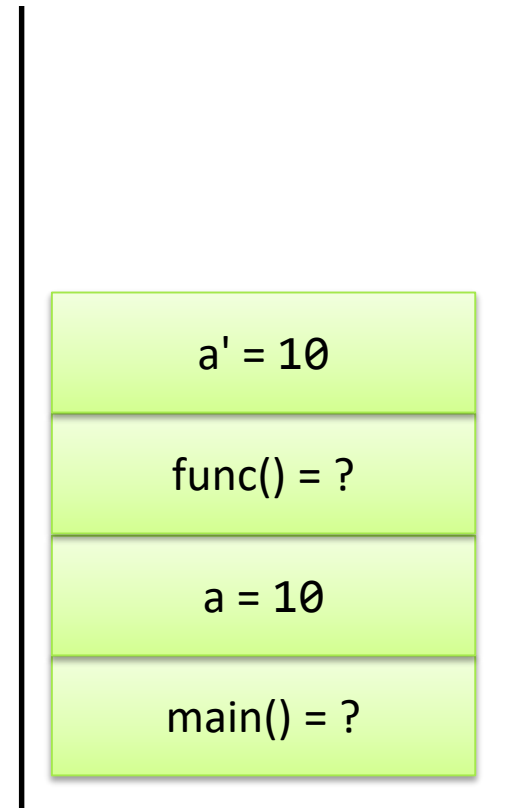
Příklad 3

```
>>> int func(int a){  
    int b = 2;  
    return b;  
}  
  
int main(){  
    int a = 10;  
    return func(a);  
}
```



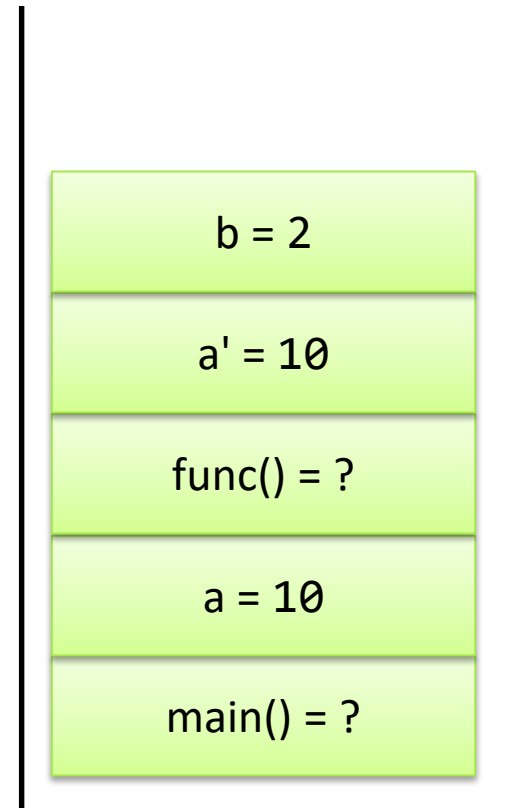
Příklad 3

```
int func(int a){  
>>>   int b = 2;  
      return b;  
}  
  
int main(){  
    int a = 10;  
    return func(a);  
}
```



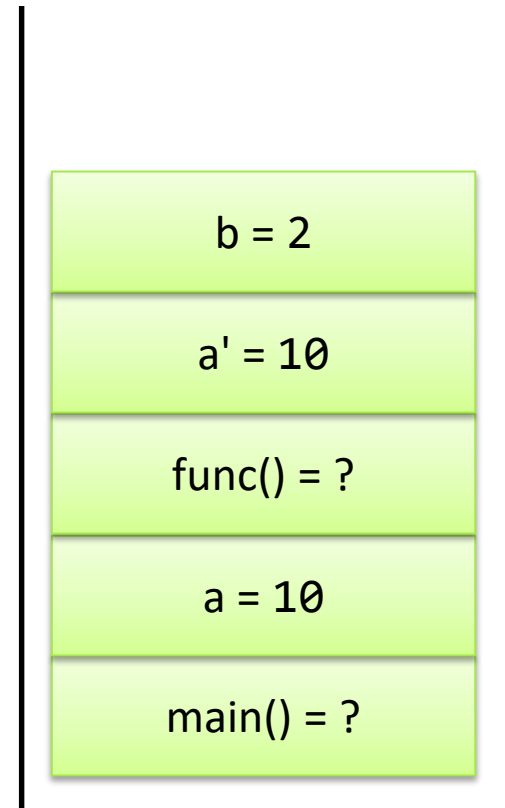
Příklad 3

```
int func(int a){  
>>>   int b = 2;  
      return b;  
}  
  
int main(){  
    int a = 10;  
    return func(a);  
}
```



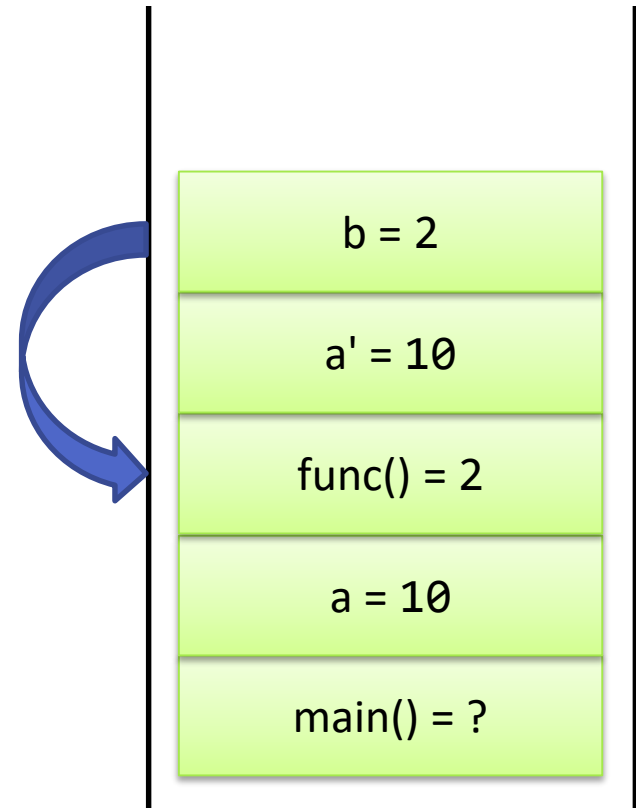
Příklad 3

```
int func(int a){  
    int b = 2;  
>>> return b;  
}  
  
int main(){  
    int a = 10;  
    return func(a);  
}
```



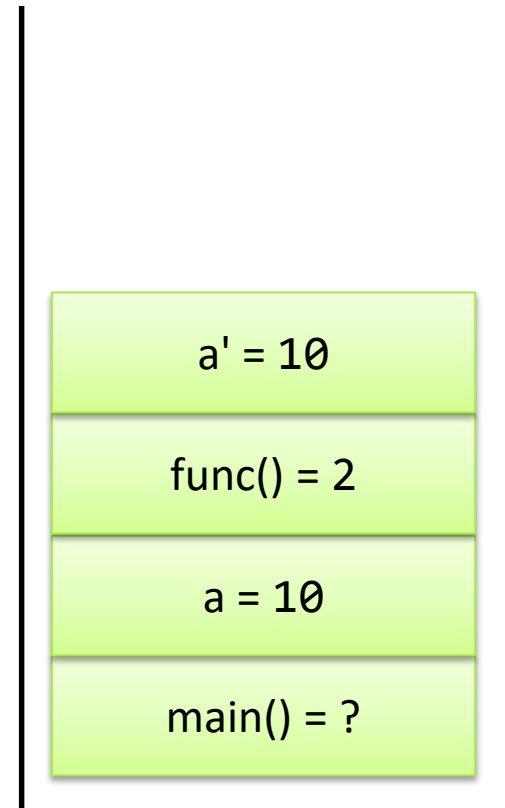
Příklad 3

```
int func(int a){  
    int b = 2;  
>>> return b;  
}  
  
int main(){  
    int a = 10;  
    return func(a);  
}
```



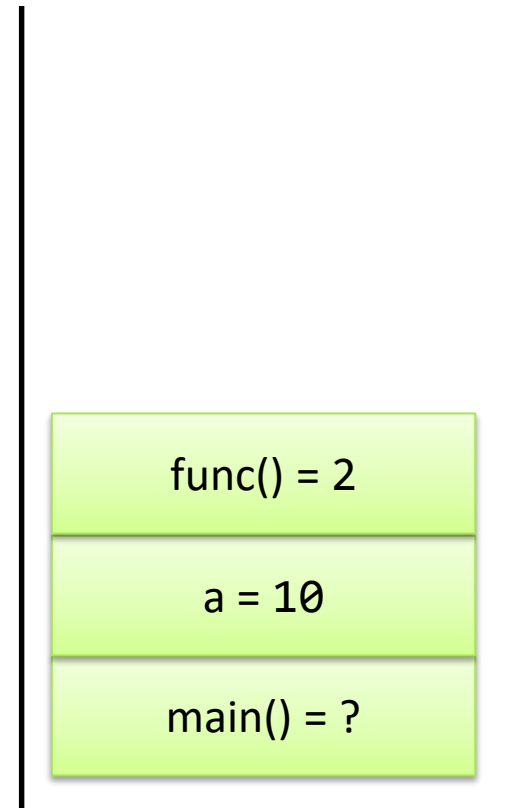
Příklad 3

```
int func(int a){  
    int b = 2;  
>>>    return b;  
}  
  
int main(){  
    int a = 10;  
    return func(a);  
}
```



Příklad 3

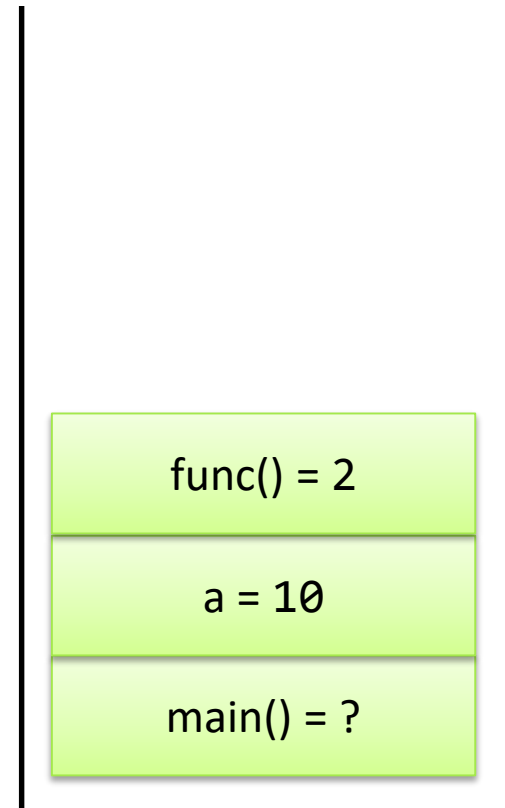
```
int func(int a){  
    int b = 2;  
>>>    return b;  
}  
  
int main(){  
    int a = 10;  
    return func(a);  
}
```



Příklad 3

```
int func(int a){  
    int b = 2;  
    return b;  
}
```

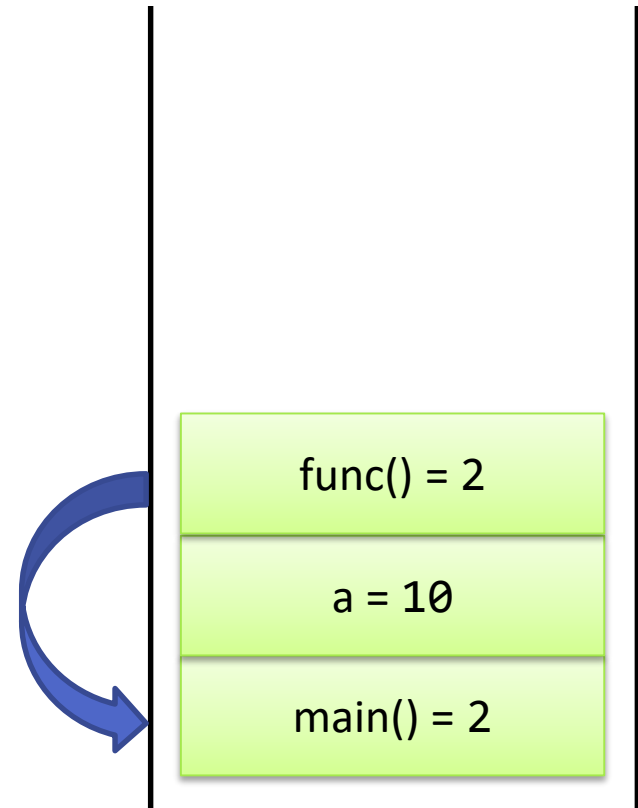
```
>>> int main(){  
    int a = 10;  
    return func(a);  
}
```



Příklad 3

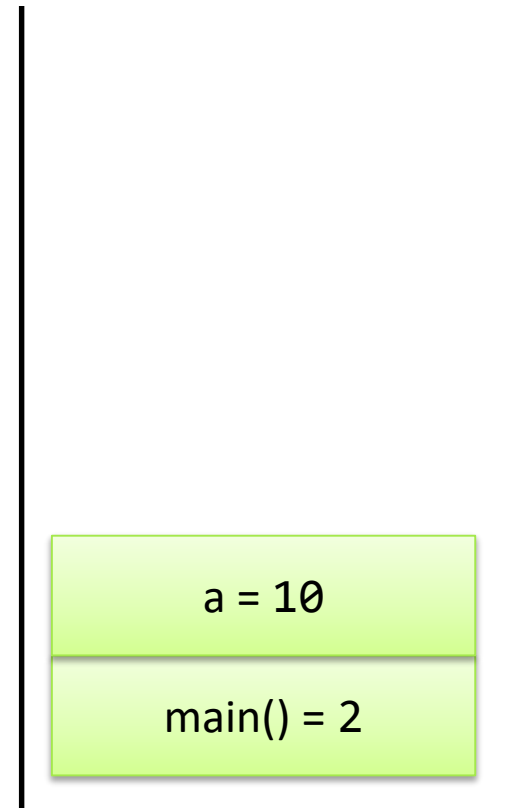
```
int func(int a){  
    int b = 2;  
    return b;  
}
```

```
>>> int main(){  
    int a = 10;  
    return func(a);  
}
```



Příklad 3

```
int func(int a){  
    int b = 2;  
    return b;  
}  
  
int main(){  
    int a = 10;  
>>>    return func(a);  
}
```



Příklad 3

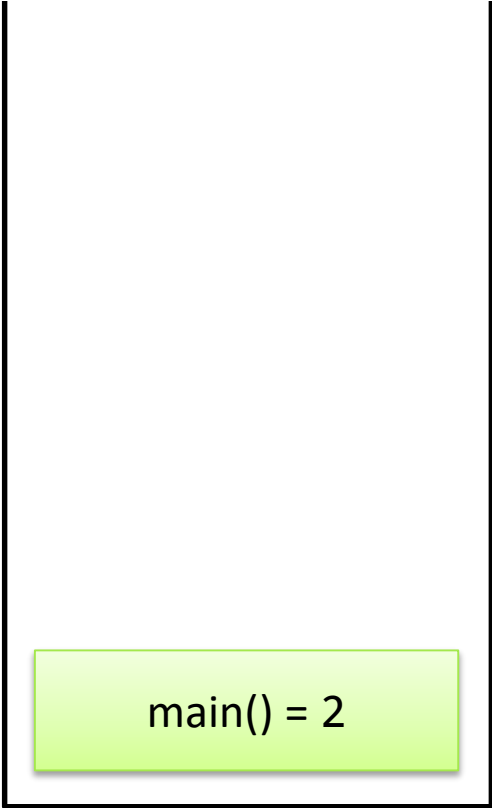
```
int func(int a){  
    int b = 2;  
    return b;  
}
```

```
>>> int main(){  
    int a = 10;  
    return func(a);  
}
```

main() = 2

Příklad 3

```
int func(int a){  
    int b = 2;  
    return b;  
}  
  
int main(){  
    int a = 10;  
    return func(a);  
}
```



main() = 2

Invariant (1/2)

- Skutečnosti, které platí po celou dobu existence objektu, nazýváme **invarianty**.
- V tomto příkladu: paměť je alokována, uživatel je přihlášen.

```
struct User {  
    User() : data(new UserData) { // získkej paměť  
        systemLogIn(*this);      // přihlas se  
    }  
    ~User() {  
        systemLogOut(*this);      // odhlas se  
        delete data;              // uvolni paměť  
    }  
private:  
    UserData* data;  
};
```

Invariant (2/2)

- Abychom zavedli invarianty, používáme:
 - konstruktory – nastolíme invarianty
 - zapouzdření pomocí viditelnosti – povolíme jen ty operace, které invarianty neporuší
 - destruktory – uklidíme, pokud je to potřeba

```
struct User {  
    User() : data(new UserData) { // získkej paměť  
        systemLogIn(*this);      // přihlas se  
    }  
    ~User() {  
        systemLogOut(*this);      // odhlas se  
        delete data;              // uvolni paměť  
    }  
private:  
    UserData* data;  
};
```

Zánik objektu

- Kdy objekt zaniká?
 - Lokální proměnná: když program opustí blok (`{ }`), ve kterém objekt vznikl.
 - Datová položka třídy `T`: když zaniká instance třídy `T`, hned po provedení destruktoru `~T`.
 - Objekt, který vznikl příkazem `new`: když zavoláme příkaz `delete` na ukazatel, který na objekt ukazuje.
 - Globální proměnná: když program opustí funkci `main`.
 - Dočasný objekt: po provedení řádku.
- Když zaniká více objektů najednou, tak zanikají přesně **v opačném pořadí**, než v jakém vznikaly.

[Příklady](#)

Příklady: pořadí vzniku a zániku (1/3)

```
struct Obj {  
    Obj(int num) : mNum(num) { std::cout << mNum << ' ' ; }  
    ~Obj() { std::cout << '~' << mNum << ' ' ; }  
private:  
    int mNum;  
};  
  
int main() {  
    Obj o1 = 1;  
    Obj o2(2);  
    Obj o3{3};  
}
```

1 2 3 ~3 ~2 ~1

Příklady: pořadí vzniku a zániku (2/3)

```
struct Obj {
    Obj(int num) : mNum(num) { std::cout << mNum << ' ' ; }
    ~Obj() { std::cout << '~' << mNum << ' ' ; }
private:
    int mNum;
};

int main() {
    Obj o0 = 0;

    for (int i = 1; i <= 3; i++) {
        Obj o = i;
    }
}
```

0 1 ~1 2 ~2 3 ~3 ~0

Příklady: pořadí vzniku a zániku (3/3)

```
struct Obj {  
    Obj(int num) : mNum(num) { std::cout << mNum << ' ' ; }  
    ~Obj() { std::cout << '~' << mNum << ' ' ; }  
private:  
    int mNum;  
};  
  
void foo(Obj o) {  
    std::cout << "foo ";  
    Obj o2 = 2;  
}  
  
int main() {  
    foo(1);  
}
```

1 foo 2 ~2 ~1

Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};
```

```
>>> int main() {  
    Osoba o1;  
    Osoba o2;  
}
```

main() = ?

Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};
```

```
>>> int main() {  
    Osoba o1;  
    Osoba o2;  
}
```

main() = ?

Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};
```

```
int main() {  
>>>     Osoba o1;  
        Osoba o2;  
}
```

o1.jmeno = ""

main() = ?

Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};
```

```
int main() {  
    >>> Osoba o1;  
        Osoba o2;  
}
```

o1.prijmeni = ""

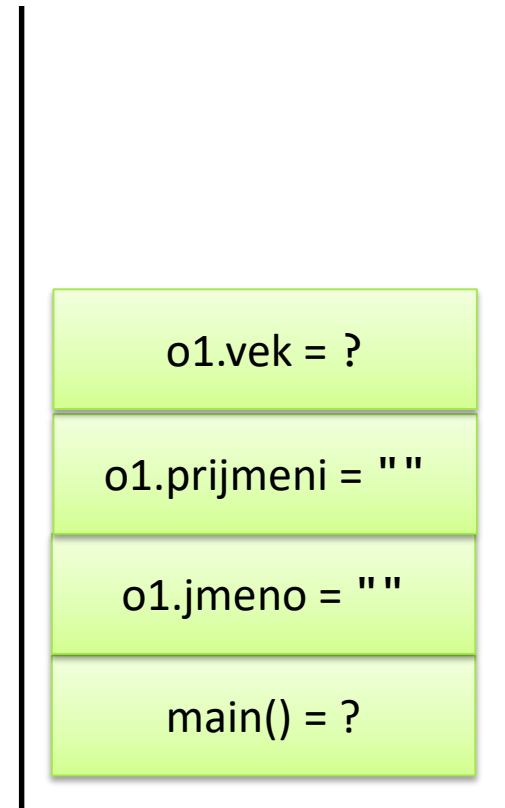
o1.jmeno = ""

main() = ?

Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};
```

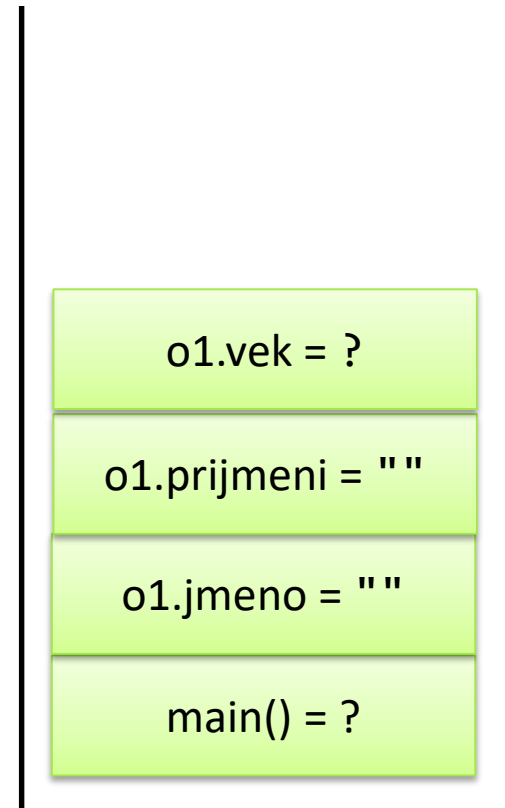
```
>>> int main() {  
    Osoba o1;  
    Osoba o2;  
}
```



Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};
```

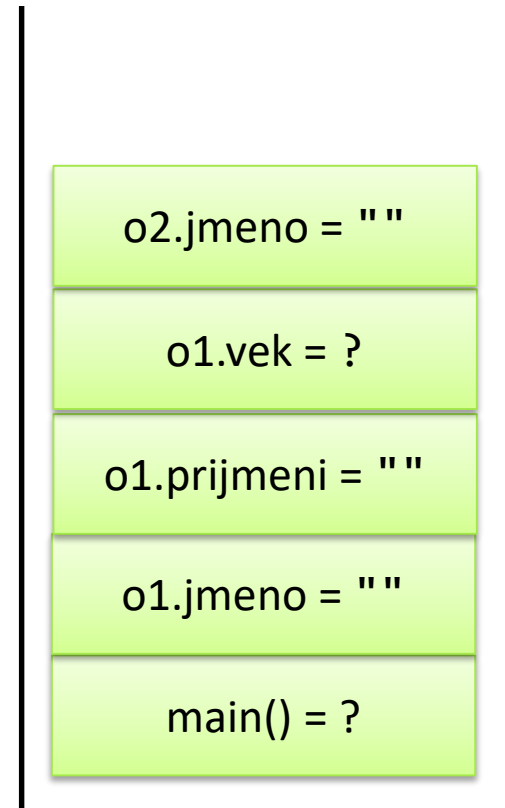
```
int main() {  
    Osoba o1;  
>>>    Osoba o2;  
}
```



Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};
```

```
>>> int main() {  
    Osoba o1;  
    Osoba o2;  
}
```



Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};
```

```
>>> int main() {  
        Osoba o1;  
        Osoba o2;  
    }
```

o2.prijmeni = ""

o2.jmeno = ""

o1.vek = ?

o1.prijmeni = ""

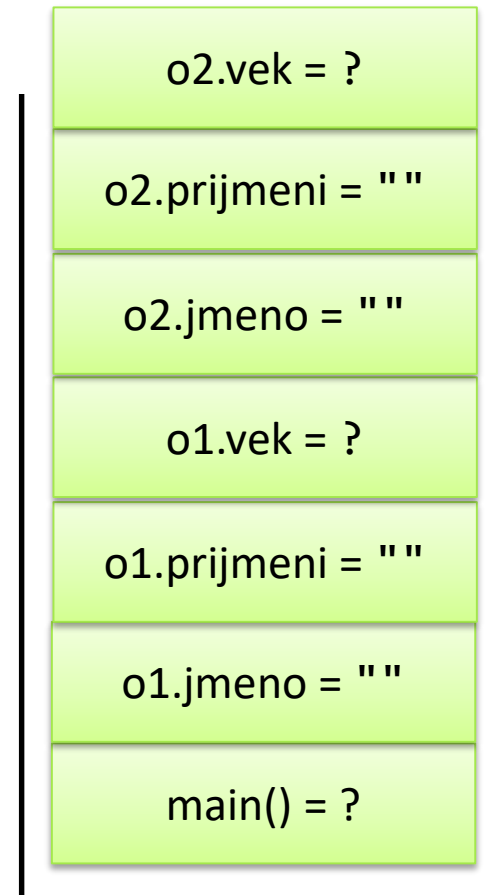
o1.jmeno = ""

main() = ?

Příklad 4

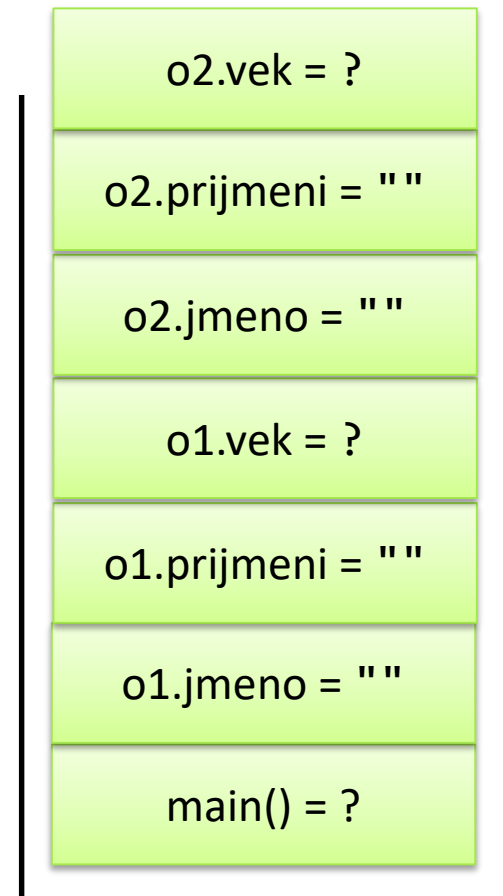
```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};
```

```
>>> int main() {  
    Osoba o1;  
    Osoba o2;  
}
```



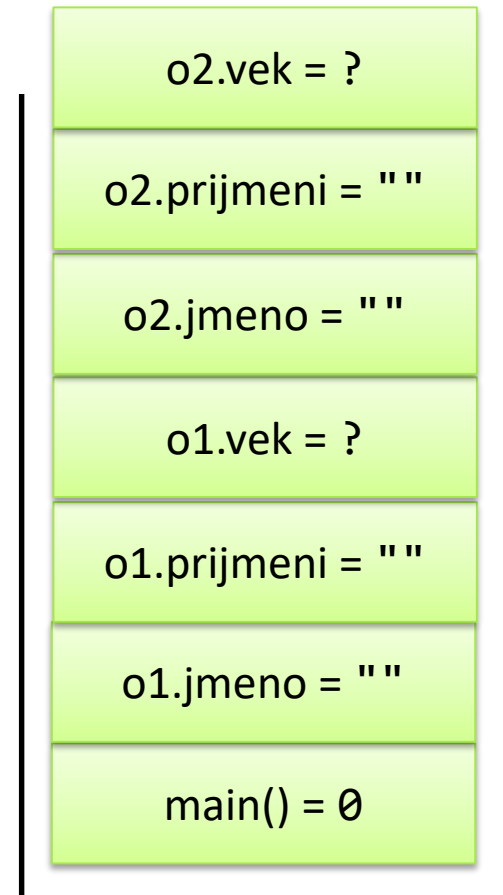
Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};  
  
int main() {  
    Osoba o1;  
    Osoba o2;  
    >>> }
```



Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};  
  
int main() {  
    Osoba o1;  
    Osoba o2;  
    >>> }
```



Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};  
  
int main() {  
    Osoba o1;  
    Osoba o2;  
    >>> }
```

o2.prijmeni = ""

o2.jmeno = ""

o1.vek = ?

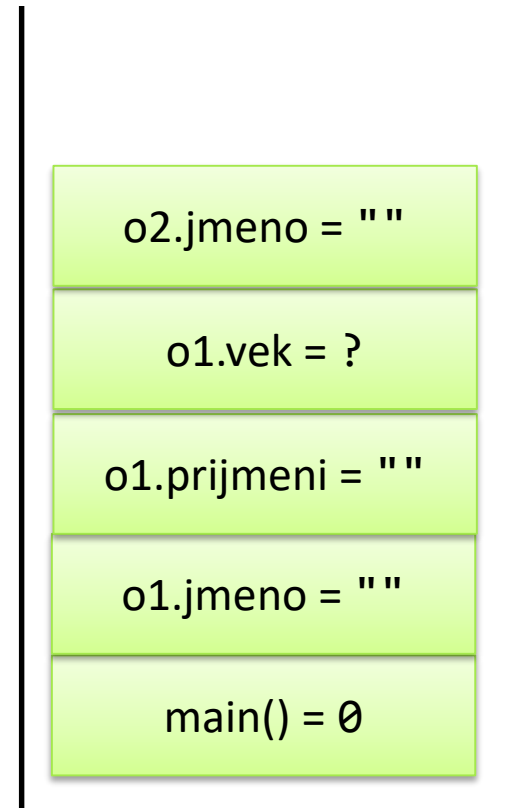
o1.prijmeni = ""

o1.jmeno = ""

main() = 0

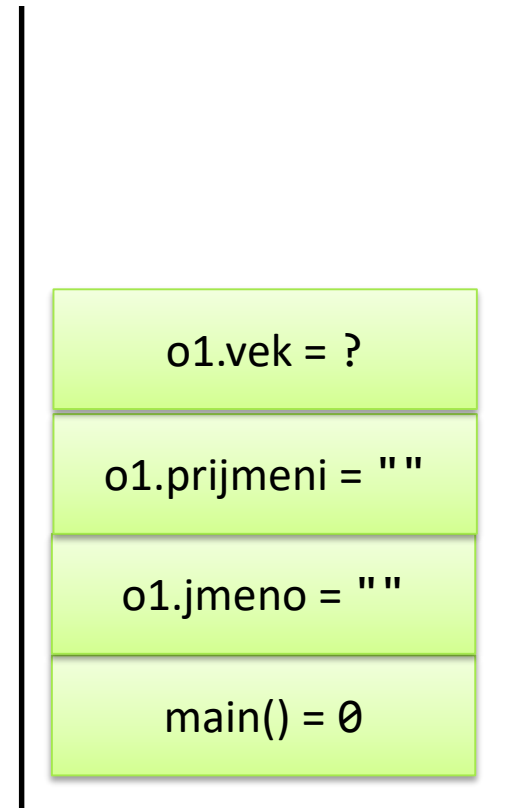
Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};  
  
int main() {  
    Osoba o1;  
    Osoba o2;  
    >>> }
```



Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};  
  
int main() {  
    Osoba o1;  
    Osoba o2;  
    >>> }
```



Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};  
  
int main() {  
    Osoba o1;  
    Osoba o2;  
    >>> }
```

o1.prijmeni = ""

o1.jmeno = ""

main() = 0

Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};  
  
int main() {  
    Osoba o1;  
    Osoba o2;  
    >>> }
```

o1.jmeno = ""

main() = 0

Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};  
  
int main() {  
    Osoba o1;  
    Osoba o2;  
    >>> }
```

main() = 0

Příklad 4

```
struct Osoba {  
    std::string jmeno, prijmeni;  
    int vek;  
};  
  
int main() {  
    Osoba o1;  
    Osoba o2;  
}
```

main() = 0

Kopírující konstruktor (*copy constructor*) (1/2)

- Konstruktor třídy T s parametrem typu **const T&** se nazývá **kopírující konstruktor** třídy T.
- Jeho úkolem je zkopírovat data do nového objektu.

```
struct Person {  
    Person();  
    Person(const std::string& name, int age);  
    Person(const Person& rhs);  
private:  
    std::string name;  
    int age;  
};  
  
Person::Person(const Person& rhs) : name(rhs.name),  
                                   age(rhs.age)  
{}
```

Kopírující konstruktor (*copy constructor*) (2/2)

- Kopírující konstruktor se zavolá v následujících situacích (o je typu Obj):

```
Obj o2 = o;  
Obj o2(o);  
Obj o2{o};  
... new Obj(o);  
... new Obj{o};
```

```
receivesObj(o);  
... Obj(o) ...;  
... Obj{o} ...;  
... : o2(o) ...
```

Příklady: kopírující konstruktor (1/2)

```
struct Obj {
    Obj(char data) : mData(data), mNum(1) {
        std::cout << "char " << mData << '-' << mNum << '\n';
    }
    Obj(const Obj& rhs) : mData(rhs.mData), mNum(rhs.mNum + 1) {
        std::cout << "copy " << mData << '-' << mNum << '\n';
    }
    ~Obj() {
        std::cout << "destr " << mData << '-' << mNum << '\n';
    }
private:
    char mData;
    int mNum;
};
```

```
int main() {
    Obj o1{'A'};
    Obj o2 = o1;
}
```

```
char A-1
copy A-2
destr A-2
destr A-1
```

Příklady: kopírující konstruktor (2/2)

```
struct Obj {  
    Obj(char data) : mData(data), mNum(1) {  
        std::cout << "char " << mData << '-' << mNum << '\n';  
    }  
    Obj(const Obj& rhs) : mData(rhs.mData), mNum(rhs.mNum + 1) {  
        std::cout << "copy " << mData << '-' << mNum << '\n';  
    }  
    ...  
};
```

```
void receivesObj(Obj o) {  
    Obj o2 = 'B';  
}  
  
int main() {  
    Obj o1{'A'};  
    receivesObj(o1);  
    Obj o3(o1);  
}
```

```
char A-1  
copy A-2  
char B-1  
destr B-1  
destr A-2  
copy A-2  
destr A-2  
destr A-1
```

Hluboká a mělká kopie (1/2)

- Kopírování v C++ je zpravidla **hluboké**. Výsledkem kopírování je tzv. **hluboká kopie** originálu.
- To znamená, že vzniklá kopie je na originálu nezávislá.
- Operace provedené na kopii originál nijak neovlivní.



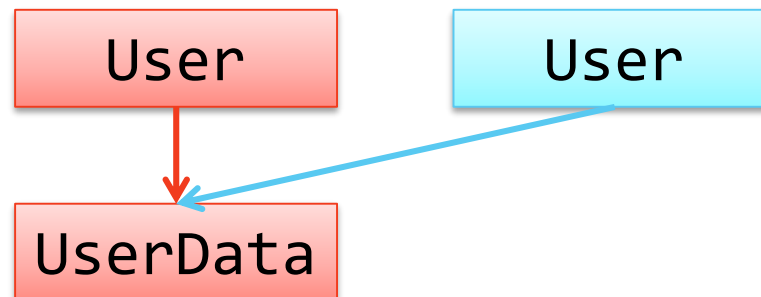
Hluboká a mělká kopie (2/2)

- Mělké kopírování v C++ neprovádíme.
- Problém mělkých kopií je, že dva a více destruktory se snaží uvolnit ty samé prostředky.
- U některých prostředků je to velká chyba, která způsobí, že program spadne.



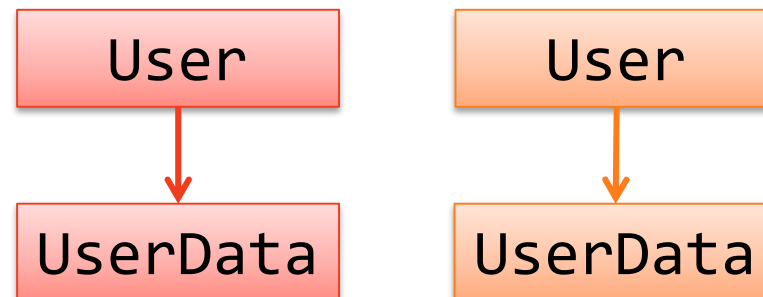
Příklad: mělká kopie

```
struct User {  
    User() : data(new UserData) {}  
    ~User() { delete data; } // destruktör uvolňuje prostředky  
    User(const User& rhs) : data(rhs.data) {} // mělká kopie  
private:  
    UserData* data;  
};  
  
int main() {  
    User u1;  
    User u2{u1}; // kopírující konstruktör  
} // chyba: dvojí smazání objektu "data", program spadne
```



Příklad: hluboká kopie

```
struct User {  
    User() : data(new UserData) {}  
    ~User() { delete data; } // destruktör uvolňuje prostředky  
    User(const User& rhs) : data(new UserData{rhs.data}) {}  
private:                                     // ^^ hluboká kopie ^^  
    UserData* data;  
};  
  
int main() {  
    User u1;  
    User u2{u1}; // kopírující konstruktor  
} // v pořádku, každý destruktör smaže vlastní data
```



Konstantní členská funkce – jen dotaz

- Zajistí, že objekt nebude měněn – jedná se o pouhý dotaz na data.
- Uvede se klíčové slovo **const** za seznam argumentů.

```
class Auto{
private: int objem; string spz;
public:
    Auto(string s, int obj) { spz = s; objem = obj; }
    Auto(){}; //I.K.
    void zobrazAuto() const;
};

void Auto::zobrazAuto() const {
    cout << "Auto: spz " << spz << " objem " << objem << endl;
}

int main() {
    Auto a1 = Auto("10-ABC-10",1500);
    Auto a2("55-AAA-55",1000);
    a1.zobrazAuto();
    a2.zobrazAuto();
    return 0;
}
```

Ukazatel this

- Každý objekt má ukazatel **this** – obsahuje adresu tohoto objektu – ukazuje na něj.
- Každá členská funkce jej může použít (skrytý argument všech metod).
- Použit při konfliktu členských dat a argumentu metody.
- Použit pro test přiřazení sebe sama.
- ***this** lze použít jako odkaz na objekt jako celek.
- Problém jak porovnat 2 objekty.

```
class Auto {  
    int objem; string spz;  
public:  
    Auto(string s, int obj) {  
        spz = s; objem = obj; }  
    int getObjem();  
    string getSpz();  
    void test(Auto&);  
    const Auto& vetsiObjem(  
        const Auto& aut) const ;  
};
```

```
int Auto::getObjem() { return(this->objem); }  
string Auto::getSpz() { return (this->spz); }  
void Auto::test(Auto & aut) {  
    if (&aut==this) cout << "Stejný objekt";  
    else cout << "Jiný objekt";  
}  
const Auto& Auto::vetsiObjem(  
    const Auto& aut) const {  
    if (aut.objem > objem) return aut;  
    else return *this;  
}
```

Konstanta platná ve třídě

- V deklaraci třídy nelze explicitně vytvořit konstantu.
- Třída popisuje objekt, ale nevytváří ho.
- Využití enumerátoru – lokální výčtový typ s jednou hodnotou.
- Využití statické proměnné.

```
class Auto{  
private:  
    enum {MAX = 5000};  
    int objem;  
    string spz;  
public:  
    Auto(const char* spz,  
        int obj);  
    int getObjem();  
};
```

```
int main() {  
    Auto a1 = Auto("10-ABC-10", 6000);  
    cout << a1.getObjem();  
    return 0;  
}  
  
int Auto::getObjem() {  
    return(this->objem);  
}  
  
Auto::Auto(string s, int obj) {  
    spz = s;  
    if (obj > MAX) objem_ = MAX;  
    else objem_ = obj;  
};
```

Pole objektů

- Důležitou strukturou při zpracování je pole objektů určitého typu.
- Objekt je uživatelský typ - proto lze použít i pole objektů.
- Při deklaraci pole – dojde k volání implicitních konstruktorů.
- Při ukončení platnosti - volání destruktoru pro každý prvek pole /pole ukazatelů.

```
class Auto{
private:
    int objem;
    string spz;
public:
    Auto(string s, int obj);
    string getSpz();
    int getObjem();
    Auto() {
        spz = „XXX-00-00“; }
    ~Auto() {
        cout << „\nrusim “<< spz;}
};
```

```
int main() {
    int ob;
    string sp;
    Auto poleAut[5];
    for (int i = 0; i<3 ; i++) {
        cout << „\nZadej spz a objem: “;
        cin >> sp >> ob;
        poleAut[i] = Auto(sp,ob);
    }
    for (int i = 0; i<3 ; i++) {
        cout<<„\n SPZ: “ <<
            poleAut[i].getSpz() <<
            “| objem: “
            << poleAut[i].getObjem();
    }
}
```

Organizace projektu v C ++

- Hlavičkový soubor – deklarace třídy, případně vložené funkce - přípona *trida.h* (header, někdy .hpp).
- Soubor *trida.cpp* se stejným jménem jako hlavička – definice třídy, implementace metod (zahr – stdafx.h).
- Soubor *projekt.cpp* s názvem projektu- tělo main a vlastní algoritmus řešení (zahr – stdafx.h).

```
//auto.h
#ifndef __AUTO_H
#define __AUTO_H
#include <iostream>
using namespace std;
class Auto {
private: int objem; string spz;
public:
    Auto(string s, int obj);
    string getSpz();
    int getObjem();
    Auto() { spz = „XXX-00-00“; }
    ~Auto() { cout << "\nrusim
"<< spz;}
};
#endif
```

```
//auto.cpp
#include "stdafx.h"
#include "auto.h"
int Auto::getObjem() { return(this-
>objem); }
string Auto::getSpz() { return(this->spz);
}
```

```
//projekt.cpp
#include "stdafx.h "
#include <iostream>
using namespace std;
#include "auto.h"
int main() {
    Auto a1 ();
    cout<<"\n SPZ: "<<a1.getSpz()<<"| objem:
"<<a1.getObjem(); return 0;}
```

Konvence C++

- Funkce **main** vrací hodnotu **int** – pro testování, 0 – v pořádku. To platí jako konvence i pro řadu knihovních funkcí.
- Identifikátor funkce – začíná malý písmenem.
- Identifikátor pojmenovaných konstant – jen velká písmena MAX_ITERATION.
- Pokud chcete, aby Váš program šlo zveřejnit - použijte angličtinu.
- Funkce a proměnné – lowerCamelCase : getName(), setValueX(int x).
- Třídy – UpperCamelCase.
- Vlastní typy – UpperCamelCase.
- Globální proměnné pokud možno nepoužívat, případně vždy přístup přes ::, nejlepší varianta – singleton.
- Soukromé položky psát se suffixem _ nebo prefixem - m_var:

```
class SomeClass { private: int length_; int m_count; }
```
- Formální argumenty u vlastních typů – stejné jméno jako typ:

```
void setTopic(Topic* topic) // NOT: void setTopic(Topic* value)
// NOT: void setTopic(Topic* aTopic)
// NOT: void setTopic(Topic* t) void
connect(Database* database) // NOT: void connect(Database* db)
// NOT: void connect (Database*
oracleDB)
```

Konvence C++ (pokr.)

Pro funkce (metody):

- **get** – získání hodnoty členských dat : getName().
- **set** – nastavení hodnoty členských dat : setName (name).
- **find** – hledání : findElement().
- **compute** – výpočet: computeAveragePrice().
- **initialize** – inicializace objektu, konceptu.
- **is** – predikát - metoda vrací boolean: isVisible(), isTrue().
- Množné číslo na skupinu objektů: int values[].
- Počet objektů – prefix n: nLines.
- Pojmenované ukazatele – Line * line (Line * pLine).
- Vyhnout se členským datům **public**.
- **NULL** je používáno v C (makro), v C++ použít **0**, nebo **nullptr** (klíčové slovo, literál, definováno C++11).

The End